Degree Project in Computer Science and Engineering

Second cycle, 30 credits

Adaptive Hybrid Retrieval-Augmented Generation

Optimizing the Computational Cost-Performance Trade-off via Learned Query Routing

DANTE WESSLUND

Adaptive Hybrid Retrieval-Augmented Generation

Optimizing the Computational Cost-Performance Trade-off via Learned Query Routing

DANTE WESSLUND

Master's Programme, Machine Learning, 120 credits
Date: August 26, 2026

Supervisors: Arvid Eriksson, Peng Zhang
Examiner: Pawel Herman
School of Electrical Engineering and Computer Science
Host company: Ericsson
Swedish title: Adaptiv hybrid-RAG
Swedish subtitle: Optimering av beräkningskostnads-prestanda-avvägningen med inlärd frågeroutning

Abstract

Retrieval-Augmented Generation (RAG) is a common approach for grounding large language models in external knowledge. Production systems increasingly combine low-cost vector retrieval with more expensive graph-enhanced retrieval, because graph-enhanced retrieval can help answer questions whose evidence is split across documents but uses substantially more compute than vector retrieval. Applying graph-enhanced retrieval to every query wastes resources on questions that the low-cost path could already handle, while always using vector retrieval risks missing questions that benefit from the richer retrieval mode.

This thesis studies whether the choice between a low-cost vector retrieval mode and a more expensive graph-enhanced retrieval mode can be predicted from the query text alone. The setting is the 2WikiMultihopQA benchmark inside the LightRAG retrieval framework, with LightRAG's Naive mode as the low-cost path and its Mix mode as the graph-enhanced path. Routing labels are constructed by running both retrieval modes on every benchmark question each of the 2,000 selected benchmark questions and using a large language model as a judge to compare each mode's answer against the benchmark reference answer. The resulting binary routing dataset (the queries for which at least one mode was judged correct) is split jointly by question type and routing label, and is used to train and evaluate two query-only routers: a TF-IDF logistic-regression baseline on term frequency–inverse document frequency (TF-IDF) features and a fine-tuned ModernBERT classifier.

On a held-out test split of 193 queries, the ModernBERT router achieves higher point estimates than the TF-IDF baseline across on the aggregate routing-label classification metrics (area the areas under the precision-recall curveand receiver operating characteristic curves, balanced accuracy, and precision, minority-class F1), with gains that , and routing-label accuracy) but a lower recall on the minority class; the estimates are stable across three training seedsbut small relative to the uncertainty of . In paired statistical tests on the same test queries, its higher precision and routing-label accuracy are significant, whereas the remaining differences are not resolved by a test set of this size. It also achieves higher F1, precision , and routing-label accuracy significantly higher precision than a non-deployable diagnostic heuristic that routes by a coarse question-category label, and higher routing-label accuracy at the unadjusted level, while using only the query text as input. Treated as offline routing policies, both learned routers route at substantially lower mean cost than always using the Mix mode: the ModernBERT router reduces mean routed execution time by 42 % and estimated mean routed model-token usage by 63 % relative to the always-Mix baseline, while retaining about 92 % of always-Mix's judged-correct answers under the stated assumption that Mix remains correct on queries where Naive was judged sufficient. These figures measure whether the router selects the judge-derived minimum sufficient retrieval mode, not an independently re-judged final answer accuracy of the routed system. Within this experimental setting, the retrieval-mode distinction is partly predictable from the query text alone, and a lightweight encoder classifier is a promising routing layer for adaptive Hybrid RAG.

Keywords

Retrieval-Augmented Generation, Hybrid RAG, Query routing

Sammanfattning

Retrieval-Augmented Generation (RAG) är en vanlig arkitektur för att förankra stora språkmodeller i externt material. I moderna system kombineras allt oftare vektorbaserad sökning med låg beräkningskostnad med dyrare grafbaserad hämtning, eftersom grafbaserad hämtning kan hjälpa till att besvara frågor vars belägg är fördelade över flera dokument men kräver väsentligt mer beräkningsresurser än vektorbaserad sökning. Att tillämpa den grafbaserade hämtningen på varje fråga slösar resurser på frågor som metoden med lägre kostnad redan klarar, medan att alltid använda enbart vektorsökning riskerar att missa frågor som drar nytta av det mer omfattande hämtningsläget.

Denna avhandling undersöker om valet mellan ett vektorbaserat hämtningsläge med låg kostnad och ett dyrare grafbaserat hämtningsläge kan förutsägas enbart utifrån frågetexten. Experimenten utförs på riktmärket 2WikiMultihopQA i hämtningsramverket LightRAG, där LightRAG:s *Naive*-läge utgör vägen med lägre kostnad och dess *Mix*-läge den grafbaserade vägen. Routingetiketter konstrueras genom att köra båda hämtningslägena på varje fråga och var och en av de 2 000 utvalda frågorna och låta en stor språkmodell som domare jämföra respektive svar mot facit. Det resulterande binära routingdatasetet (frågorna där minst ett hämtningsläge bedömdes korrekt) delas upp gemensamt efter frågetyp och routingetikett och används för att träna och utvärdera två routrar som endast ser frågetexten, nämligen en TF-IDF-baserad logistisk regressionsmodell logistisk regressionsmodell på TF-IDF-representationer (term frequency–inverse document frequency) som baslinje och en finjusterad ModernBERT-klassificerare.

På en testmängd om 193 frågor uppnår ModernBERT-routern högre punktestimat än TF-IDF-baslinjen på de aggregerade routingklassificeringsmåtten (arean under precision-recall-kurvanareorna under precision-recall- och ROC-kurvorna, balanserad noggrannhetoch , precision, F1 på minoritetsklassen), med förbättringar som och routingnoggrannhet) men lägre recall på minoritetsklassen, och estimaten är stabila över tre träningsfrönmen små i förhållande till osäkerheten hos . I parade statistiska test på samma testfrågor är routerns högre precision och routingnoggrannhet signifikanta, medan övriga skillnader inte kan avgöras med en testmängd av denna storlek. Routern uppnår även signifikant högre F1, precision och routingnoggrannhet precision än en icke-driftsättningsbar diagnostisk heuristik som ruttar utifrån en grov frågekategorietikett, samt högre routingnoggrannhet före justering för multipla jämförelser, samtidigt som den endast använder frågetexten som indata. Som offline-routingpolicyer ger båda routrarna betydligt lägre medelkostnad än att alltid använda Mixläget: ModernBERT-routern minskar medelexekveringstiden med 42 % och den uppskattade användningen av modelltoken med 63 % relativt always-Mix-baslinjen, samtidigt som den behåller omkring 92 % av always-Mix:s bedömt korrekta svar under antagandet att Mix förblir korrekt på frågor där Naive bedömts tillräcklig. Dessa mått avser huruvida routern väljer den bedömda minsta tillräckliga hämtningsmetoden, inte en separat ombedömd slutlig svarskorrekthet för hela routingsystemet. I denna experimentella miljö är valet av hämtningsläge delvis förutsägbart enbart utifrån frågetexten, och en lättviktig encoder-klassificerare är ett lovande routinglager för adaptiv hybrid-RAG.

Nyckelord

Retrieval-Augmented Generation, hybrid-RAG, frågeroutning

Acknowledgments

I would like to thank my KTH supervisor Arvid Eriksson and my Ericsson supervisor Peng Zhang for their guidance, feedback, and continuous support throughout this thesis project. The use of generative AI tools during this project is reported in Section 5.7.

Stockholm, August 2026
Dante Wesslund

Contents

List of Figures

List of Tables

List of acronyms and abbreviations

API	application programming interface
AUPRC	area under the precision-recall curve
AUROC	area under the receiver operating characteristic curve
BERT	Bidirectional Encoder Representations from Transformers
CPU	central processing unit
GPU	graphics processing unit
JSON	JavaScript Object Notation
LLM	large language model
LLM-as-a-Judge	large language model as a judge
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
TF-IDF	term frequency-inverse document frequency
UN	United Nations

Chapter 1

Introduction

This chapter introduces the thesis topic and motivates the problem studied in this thesis project. Section 1.1 gives the necessary background in retrieval-augmented question answering and the cost of different retrieval modes. Section 1.2 defines the problem and research question. Section 1.3 states the purposeof the work, Section 1.4 presents the goals, Section 1.5 outlines the research methodology, Section 1.5 defines the , states the research question, purpose, goals, and delimitations, and Section 1.6 describes the structure ends with an outline of the rest of the thesis (Section 1.6).

1.1 Background

Retrieval-Augmented Generation (RAG) has become a common architecture for grounding large language model (LLM) outputs in external documents, especially when the relevant information is private, domain-specific, or changes after model pretraining [1]. A typical RAG system retrieves text passages from a corpus and conditions the generator on those passages, making the language model less dependent on memorized knowledge.

The simplest retrieval layer is vector similarity search over document chunks. It is efficient and often effective for local factual questions, but it can struggle when a query depends on linking evidence across documents, resolving relationships between entities, or comparing distributed attributes. Multi-hop question answering benchmarks are designed around exactly this regime, in which the answer can only be derived by chaining several supporting facts together. Graph-enhanced RAG systems address this by introducing structured representations of entities, relations, or higher-level document organization. Recent systems such as GraphRAG and LightRAG show how graph-based structure can support retrieval beyond vector similarity chunks

[2, 3].

Richer retrieval, however, incurs additional computational cost. Graph construction, graph traversal, larger context assembly, and additional language-model calls all add to the cost of retrieval. At Ericsson, the host company motivating this thesis, where internal technical repositories are large and span many documents, graph-enhanced retrieval may improve answer quality for difficult questions, but applying it to every query can waste computational power on questions that lower-cost vector retrieval already handles [4, 5].

This tension motivates adaptive retrieval. Self-RAG and Adaptive-RAG argue that retrieval behavior should depend on the query rather than follow a fixed policy [4, 6]. The same cost-performance reasoning appears in *model routing*, where a router decides for each query whether a powerful but computationally expensive language model is needed or whether a smaller, less expensive one would be sufficient [5].

The question studied here is narrower than general model routing. Instead of routing between different language models, the project routes between retrieval modes inside a fixed Hybrid RAG setting. The low-cost path is LightRAG's *Naive* mode, which uses only vector retrieval, and the high-cost path is LightRAG's *Mix* mode, which combines vector retrieval with graph-enhanced retrieval. The routing layer is intentionally lightweight, because if the router itself requires a computationally expensive model, much of the intended efficiency gain is lost.

1.2 Research problem

The problem approached in this thesis is whether the choice between a low-cost vector-based retrieval path and a more expensive graph-enhanced retrieval path in Hybrid RAG can be made before retrieval runs, from the query text alone. In LightRAG, this is operationalized as a choice between *Naive* retrieval and *Mix* retrieval. Prior work motivates both graph-based RAG and adaptive retrieval [2, 3, 4, 6], but does not quantify how well this retrieval-mode choice can be predicted in advance inside a fixed Hybrid RAG framework.

1.2.1 Problem definition

The research problem is to decide, for a given query, whether the low-cost vector retrieval mode is judged sufficient or whether the more expensive graph-enhanced retrieval mode is judged beneficial under the experimental pipeline.

The scientific issue is whether this judged retrieval need is predictable from the query alone. A router that sees only the query text can succeed only if the need is encoded in how the question is phrased. If the need is determined mainly by what the retriever surfaces after retrieval has already run, no query-only router can anticipate it.

The label studied here is operational. A query labeled `mix_required` means that, under the chosen LightRAG configuration and judge prompt, *Mix* was judged correct when *Naive* was not. The label captures whether *Mix* was beneficial under the experimental pipeline, without identifying which component of *Mix* (graph traversal, context assembly, or retrieval breadth) drove the improvement.

1.2.2 Research question

Within a fixed Hybrid RAG setting, how well can a lightweight query-only classifier predict a judge-derived routing label for the minimum sufficient retrieval mode, and how does using that classifier as an offline routing policy affect routed cost relative to static *Naive* and *Mix* baselines?

The research question is further split into the following sub-questions:

- How well does a fine-tuned encoder-only classifier predict the judge-derived routing label, compared compare to a term frequency-inverse document frequency (TF-IDF) logistic-regression baseline and to a non-deployable coarse question-category heuristic in predicting the judge-derived routing label?

- When classifier scores are turned into a thresholded routing policy, how much routed execution time and estimated model-token cost are saved relative to always-*Mix* at the selected routing-label operating point?

1.3 Purpose

The purpose of this work is to reduce the computational cost of Hybrid RAG retrieval by making the choice of retrieval mode adaptive, so that the more expensive graph-enhanced mode is reserved for the queries that need it rather than applied to every query. Because a routing layer is only worthwhile if it is itself cheap, a second purpose is to test whether that decision can be made from the query text alone with a lightweight classifier, instead of with an additional expensive model call. In a setting such as Ericsson's, where retrieval runs over large internal corpora, even a partial reduction in how often the expensive mode is invoked translates into a meaningful saving at scale.

1.4 Goals

The main goal is to design and evaluate a learned routing layer that predicts whether a query should use LightRAG's *Naive* or *Mix* retrieval mode. Four concrete objectives structure the work.

1. Implement a reproducible pipeline for preparing samples from a multi-hop question-answering benchmark, indexing them in LightRAG, and executing both *Naive* and *Mix* retrieval modes.

2. Construct a supervised routing dataset by labeling each query according to the minimum judged sufficient retrieval mode, using a large language model as a judge (LLM-as-a-Judge) to compare retrieval outputs against benchmark answers.

3. Train lightweight query-only routers, including a ModernBERT-based classifier and a classical TF-IDF logistic-regression baseline, to predict the *Naive*-sufficient versus *Mix*-beneficial routing label.

4. Evaluate the routers both as held-out classifiers, using routing-label metrics such as area under the precision-recall curve (AUPRC) and F1 on the *Mix*-beneficial class, and as thresholded offline routing policies measured against always-*Naive*, always-*Mix*, and a non-deployable diagnostic heuristic that routes by a coarse question-category label, on mean routed execution time and estimated mean routed model-token usage.

This thesis is a quantitative empirical study on a public multi-hop question-answering benchmark. The predictability of retrieval-mode selection from the query text is evaluated by training query-only routers on judge-derived routing labels and obtaining classification metrics on held-out data. These include AUPRC, balanced routing-label accuracy, and F1 on the minority class. The same routers are additionally evaluated as thresholded offline routing policies and compared against the static always-*Naive* and always-*Mix* baselines along routed-cost metrics. A non-deployable diagnostic heuristic that routes by a coarse question-category label is included as an additional reference, to measure how much of the routing signal is already captured by such a categorical rule. Bootstrap confidence intervals are used to estimate the uncertainty of these metrics. Comparisons between the learned routers, the static baselines, and the categorical heuristic are then used to answer the research question.

The reported classification metrics measure agreement with the judge-derived routing label. They are distinct from final-answer accuracy of the selected retrieval path. Routing a *Naive*-sufficient query to *Mix* is counted as a routing-label error because the lower-cost mode was sufficient, although the *Mix* answer for that query may still be correct. This distinction matters when interpreting the static always-*Mix* baseline and the routed-cost results in Chapter 4.

1.5 Delimitations

The scope is deliberately limited to query-only retrieval-mode routing. The router receives the query text as input and does not inspect retrieved documents, intermediate graph state, answer drafts, or model uncertainty after generation. This keeps the routing problem inexpensive and well-defined, but it means the project does not evaluate routers that adapt after retrieval has already begun.

The retrieval setting is LightRAG with two modes, *Naive* and *Mix*. These operationalize the broader distinction between vector retrieval and graph-enhanced retrieval, but results should not be assumed to transfer automatically to every Hybrid RAG framework.

The primary benchmark is 2WikiMultihopQA [7], chosen because it is public, provides supporting evidence, and makes the experiment reproducible without depending on confidential data. Ericsson documentation motivates the industrial problem but is not the core benchmark.

Routing labels come from LLM-as-a-Judge and are treated as judged supervision rather than absolute ground truth. Cases where neither retrieval mode produces a correct answer are excluded from router training.

The offline policy evaluation does not re-judge newly generated answers, because it reuses precomputed *Naive* and *Mix* outputs. Unless stated otherwise, reported accuracy values refer to routing-label accuracy. The two routing-error types are reported separately as the under-routing rate (routing a `mix_required` query to *Naive*) and the over-routing rate (routing a `naive_enough` query to *Mix*), and they have different practical weight, as defined in Section 3.5.2 and discussed in Chapter 44.

1.6 Structure of the thesis

The remainder of the thesis is organized as follows. Within this chapter, Section 1.1 gives the necessary background in retrieval-augmented question answering and the cost of different retrieval modes, Section 1.2 defines the problem and the research question, Section 1.3 states the purpose of the work, Section 1.4 presents the goals, and Section 1.5 defines the delimitations.

Chapter 2 presents the technical and scientific background on RAG, graph-enhanced retrieval, adaptive retrieval, query routing, ModernBERT, and related work. Chapter 3 describes the experimental method, including method: data preparation, LightRAG execution, label generation, model training, and evaluation designthe router models and their training, the evaluation metrics and statistical tests, and the design of the experiments. Chapter 4 describes the implementation, presents the held-out classification and offline routed-cost experiments, reports the results, and discusses what they imply about the predictability of retrieval-mode selection4 reports and analyses the results of these experiments, finding by finding. Chapter 5 discusses the findings in the context of prior work, their industrial and societal relevance, ethical and sustainability aspects, and the limitations of the study, and documents the use of generative artificial intelligence (AI) tools during the project. Chapter 6 concludes with answers to the research question and suggestions directions for future work.

Chapter 2

Background

This chapter introduces the technical and scientific concepts that the rest of the thesis builds on. Section 2.1 describes RAG, why it is needed for grounding large language model outputs, and how the components fit together. Section 2.2 contrasts vector retrieval with graph-enhanced retrieval, including the two LightRAG retrieval modes used in this thesis. Section 2.3 describes adaptive retrieval and query routing. Section 2.4 introduces the language models and text classifiers used by the router. Section 2.5 positions the thesis in relation to prior work.

2.1 Retrieval-augmented generation

Large language models encode an enormous amount of general knowledge in their parameters, but they have two limitations relevant to industrial question answering. They cannot access information that was not part of their training data, which makes them unable to answer questions over private or company-internal documents, and they have a tendency to hallucinate, generating fluent but factually incorrect answers when asked about content outside their training distribution [1]. RAG addresses both problems by retrieving relevant passages from an external corpus at query time and conditioning the language model on those passages, which both gives the model access to information beyond its training data and lets it ground its answers in citable sources [1, 8].

The original RAG formulation by Lewis et al. [8] treats retrieval and generation jointly as a latent-variable problem. Given a query q, a retriever returns a set $\mathcal{D}_k(q)$ of the k most relevant documents from the corpus, and the generator produces an answer a by marginalizing over which document was retrieved, as in

$$p(a \mid q) = \sum_{d \in \mathcal{D}_k(q)} p_\theta(a \mid q, d)\, p_\eta(d \mid q), \tag{2.1}$$

where $p_\eta(d \mid q)$ is the retriever (parameters η) assigning a probability to each candidate document given the query, and $p_\theta(a \mid q, d)$ is the generator (parameters θ) producing the answer given the query and a retrieved document. Many production RAG systems use simpler decoding procedures than this marginalization, but Equation 2.1 makes the point that matters here explicit. The documents that the retrieval system surfaces directly determine what the generator can answer.

For industrial question answering the motivation is concrete. At Ericsson, internal technical documents cannot be assumed to appear in any public language-model pretraining corpus, so a RAG system is needed to ground answers in company-controlled knowledge. The same is true of many other domain-specific settings, which is why RAG has become a standard architecture for grounded question answering over private corpora [1].

Many real questions cannot be answered from a single passage. Multi-hop question answering benchmarks such as HotpotQA, 2WikiMultihopQA, and MuSiQue are constructed around questions whose answers require combining information from multiple supporting documents [7, 9, 10]. For example, a question of the form *"Who is the spouse of the director of film X?"* requires the system first to identify the director of X, then to look up that person, and finally to retrieve their spouse. A retriever that scores each document independently against the query may surface only part of the chain.

2.2 Graph-enhanced retrieval

Most practical RAG systems start with vector retrieval. Documents are split into chunks, each chunk is encoded into a dense vector by an embedding model, and at query time the chunks whose vectors are closest in the embedding space to the query's vector are returned to the generator [8, 11]. This is fast and works well when the answer is contained inside a single passage that resembles the query.

Vector retrieval is weaker when the required evidence is split across multiple documents or depends on relationships between entities. Closeness in the embedding space measures local textual similarity but does not capture the structure that links documents together. Matching the query against individual chunks can therefore surface passages that are each relevant on their own and still miss the full answer chain.

2.2.1 Graph-based RAG systems

Graph-based RAG systems address this limitation by constructing a graph over the corpus before any query is asked. Nodes correspond to entities, chunks, or higher-level summaries, and edges correspond to relations between them. At query time the system can then retrieve evidence by traversing this graph rather than only matching the query against individual chunks.

GraphRAG [2] extracts entities and their relations from each chunk using a language model, builds a knowledge graph over the entire corpus, and groups closely related entities into communities. Each community is summarized into a higher-level description, which can be used to answer queries that span large parts of the corpus. LightRAG [3] follows a similar two-level idea but is designed to be faster and easier to deploy. It extracts an entity-relation graph during indexing, combines it with vector chunks at retrieval time, and exposes several retrieval modes that vary in how heavily they use the graph. LightRAG is the framework used in this thesis because it exposes a low-cost vector-only retrieval mode and a more expensive graph-enhanced mode inside the same system, which makes the two modes directly comparable on identical inputs.

Graph-enhanced retrieval is significantly more computationally expensive than pure vector retrieval. Building the graph requires many language-model calls during indexing for entity and relation extraction, and answering a query involves traversing the graph, assembling a larger context, and often additional language-model calls. Microsoft's cost discussion for GraphRAG identifies these repeated language-model calls as the dominant cost driver [2, 12]. Graph-enhanced retrieval is therefore best treated as an expensive capability that should be invoked only when the question benefits from it.

2.2.2 LightRAG *Naive* and *Mix* modes

LightRAG exposes several retrieval modes, two of which are used as the routed paths in this thesis [3]. The *Naive* mode retrieves the top-k text chunks for the query by vector similarity and passes them directly to the generator, ignoring the graph. The *Mix* mode is hybrid: it retrieves vector chunks as in *Naive*, additionally retrieves entities and relations from the entity-relation graph, and assembles both kinds of evidence into the generator's context. The two modes share the same underlying corpus and generator but differ in how much graph structure they use, which makes them a natural pair of routed retrieval paths.

Figure 2.1 illustrates the difference with a stylized multi-hop example. The *Naive* mode retrieves chunks similar to the query and can fail when no single chunk contains both required hops. The *Mix* mode follows entity-relation edges in the graph and can connect the two hops even when they live in separate documents.

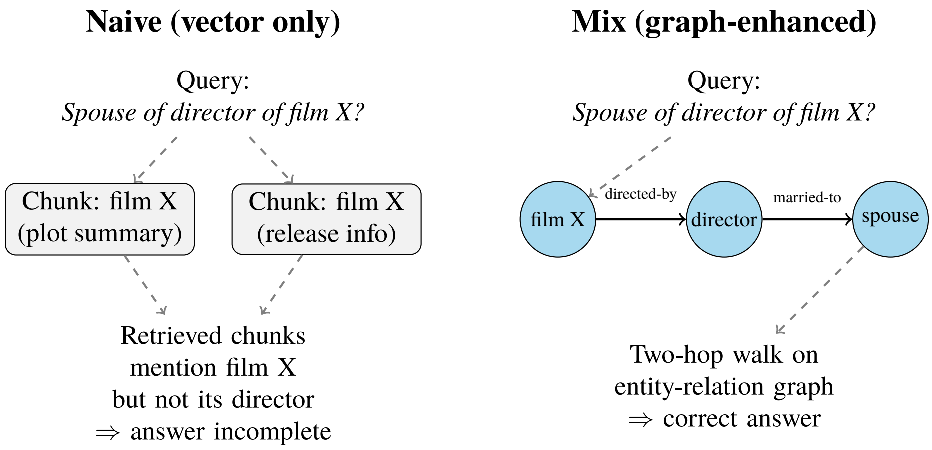

Figure 2.1: Stylized illustration of vector retrieval (*Naive*) versus graph-enhanced retrieval (*Mix*) on a two-hop question. Vector retrieval matches chunks individually against the query, while the graph-enhanced mode can follow entity-relation edges that connect facts living in different documents.

2.3 Adaptive retrieval and query routing

Adaptive retrieval is motivated by the fact that different queries benefit from different retrieval strategies. Self-RAG [6] fine-tunes the generator to emit reflection tokens that decide whether retrieval is necessary, what to retrieve, and whether the generated answer is supported by the evidence, which lets the system skip retrieval entirely on questions it is confident about. Adaptive-RAG [4] takes a different approach: a small classifier estimates the complexity of a question and the system uses that estimate to select between progressively more expensive retrieval-augmented reasoning strategies, ranging from no retrieval through single-hop retrieval to iterative retrieval. Both works support the premise that retrieval behavior should depend on the query, and Adaptive-RAG in particular shows that a small upstream classifier can drive the strategy choice.

Model routing frames the same idea more generally as a cost–quality decision [5]. In a multi-model setting, the system has access to several language models with different cost and accuracy profiles, and a router decides on a per-question basis whether to use a powerful but expensive model or a smaller, less expensive one. RouterBench [5] is a benchmark and evaluation suite for this problem and reports that even simple routers can recover a large fraction of the strongest model's accuracy while spending only a fraction of its cost. Although RouterBench concerns routing between language models rather than between retrieval modes, the abstraction transfers directly. A router observes an input and chooses among execution paths with different cost and expected performance before the expensive computation happens.

This thesis applies that routing abstraction inside the retrieval layer, selecting between two retrieval modes within one Hybrid RAG framework. Figure 2.2 illustrates the routing decision. Adaptive-RAG and similar work likewise make their routing decision before retrieval runs, on the query alone [4, 5].

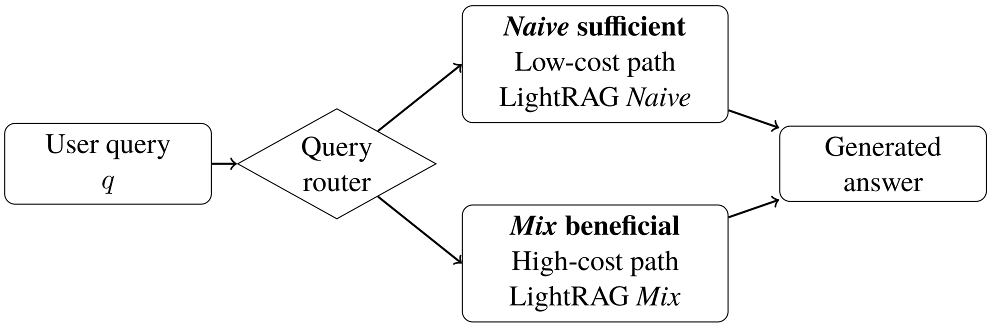

Figure 2.2: Query routing between LightRAG's low-cost *Naive* path and its more expensive *Mix* path. The router observes only the query and selects a retrieval mode before retrieval runs.

2.4 Language models and text classifiers

The routing layer is implemented as a text classifier over the query. This section describes the two classifier families compared in this thesis, a transformer-based encoder model (ModernBERT) and a classical TF-IDF logistic-regression baseline.

2.4.1 Encoder-only transformers and ModernBERT

The transformer architecture [13] introduced self-attention as a general mechanism for contextualized text representations. Encoder-only models such as Bidirectional Encoder Representations from Transformers (BERT) are trained on masked language modeling and produce a contextualized vector per input token, which can be pooled or fed to a small classification head to produce a class label [14]. Decoder-only models such as the GPT (generative pre-trained transformer) family are instead trained to predict the next token and are more naturally used as generative language models.

Both families can in principle be used for classification, but encoder-only models remain attractive when the output is a class label and inference efficiency matters. A recent comparison by Benayas et al. [15] finds that encoder-only models match or exceed much larger decoder-only models on intent classification and sentiment analysis while being orders of magnitude smaller, which is consistent with the design goal of a low-cost routing layer.

ModernBERT [16] is a recent encoder model that updates the BERT architecture with modern training data, longer context, and several efficiency improvements. The authors report that ModernBERT-base matches or exceeds the classification quality of much larger models while remaining light enough to fine-tune on a single graphics processing unit (GPU), which makes it a natural choice for a query-only router that has to remain inexpensive relative to the retrieval modes it routes between.

2.4.2 TF-IDF logistic regression

The classical baseline used in this thesis is a logistic regression over TF-IDF features [17]. Each query is represented by a sparse vector whose entries are term frequencies weighted by the inverse document frequency of the term in the training corpus, which downweights very common words. A logistic regression then maps this sparse feature vector to a probability that the query is *Mix*-beneficial under the judged setup. The baseline matters because the distinction between *Naive*-sufficient and *Mix*-beneficial queries may be partly lexical. Multi-hop questions often have characteristic surface patterns. Nested relations like *"the spouse of the director of X"* signal a chained lookup, and comparisons between two entities signal that evidence must come from multiple sources. A TF-IDF classifier can pick up such patterns from word frequencies alone, so it sets a meaningful floor for how much a pretrained transformer encoder needs to add.

2.5 Related work

The most closely related prior work spans graph-based RAG systems, adaptive-retrieval work that uses an upstream classifier, and general model routing.

Edge et al. [2] introduce GraphRAG, and Guo et al. [3] introduce LightRAG. Both systems demonstrate that graph-enhanced retrieval can add structure beyond local vector similarity and improve answer quality on questions that require linking facts across documents. They also document the additional cost of doing so. Neither work addresses the follow-up question of how the system should decide which retrieval mode to use for a given query. This thesis treats that selection as a supervised prediction problem.

Jeong et al. [4] present Adaptive-RAG, the closest precedent on the methodological side. They train a small classifier on the question to estimate complexity and use that estimate to select between progressively more expensive retrieval-augmented reasoning strategies, ranging from no retrieval to iterative retrieval. The classifier is trained on labels derived from how well different strategies answer the question on a training set, which is conceptually similar to the judge-derived labels used here. Adaptive-RAG and this thesis differ in two respects. Adaptive-RAG selects across these reasoning strategies at the language-model level, whereas this thesis selects between LightRAG's *Naive* and *Mix* retrieval modes inside one fixed framework. The classifier here is also intentionally restricted to a lightweight encoder so the routing cost remains negligible. Asai et al. [6] achieve a similar adaptive behavior with Self-RAG by training the generator itself to emit reflection tokens, which is a more invasive change and does not give an explicit pre-retrieval routing decision.

Hu et al. [5] formalize routing between language models as a cost and quality trade-off and provide RouterBench as a benchmark for it. Although RouterBench routes between language models, the framing carries over directly to retrieval-mode routing. The trade-off plots used in Chapter 4 4 follow the same logic. Each operating point of the learned router can be compared to static baselines, and the question of interest is whether the router achieves useful routing-label performance at substantially lower routed cost than the always-expensive policy.

This To the best of the author's knowledge, the reviewed literature does not contain a focused empirical study of query-only routing between a low-cost vector retrieval mode and a more expensive graph-enhanced retrieval mode inside a single Hybrid RAG framework, treated as a supervised classification problem with judge-derived labels. This thesis addresses that gap, using LightRAG's *Naive* and *Mix* modes as the concrete low-cost and graph-enhanced paths.

Chapter 3

Methods

This chapter describes the data, the routing routing-label pipeline, the router models , and and how they are trained, the evaluation procedureused to study query-only retrieval-mode routing, and the design of the experiments whose results are reported in Chapter 4. Section 3.1 describes the benchmark data and how it is preprocessed and indexed. Section 3.2 describes the routing-label pipeline that turns raw benchmark questions into a supervised routing dataset. Section 3.3 describes the resulting binary routing dataset and the held-out splits (the validation and test splits, neither of which is used to fit any model parameters) used for evaluation. Section 3.4 describes the two router models, a ModernBERT-based classifier and a TF-IDF logistic-regression baseline, together with their implementation and training procedure. Section 3.5 describes defines the held-out classification metricsand , the offline routing-policy evaluation. , and the statistical tests used to compare the routers. Section 3.6 describes the experiments.

Figure 3.1 gives a visual overview of the full pipeline; the sections that follow describe each step in turn.

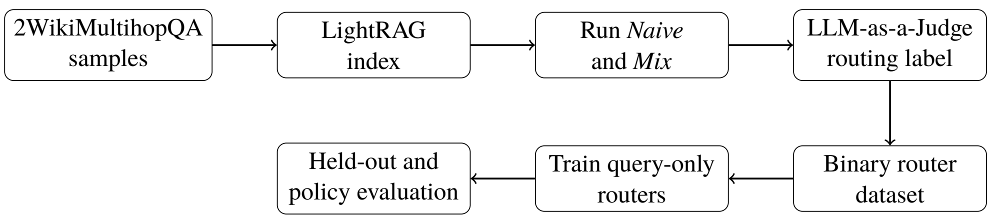

Figure 3.1: End-to-end pipeline from 2WikiMultihopQA samples to router evaluation. Each step writes its output to disk so the remaining steps can be re-run without repeating expensive earlier steps.

3.1 Data

This section describes the 2WikiMultihopQA benchmark (Section 3.1.1) and the preprocessing and indexing steps that prepare it for the routing-label pipeline (Section 3.1.2).

3.1.1 2WikiMultihopQA

The benchmark used in this thesis is 2WikiMultihopQA [7], a public multi-hop question answering dataset constructed from Wikipedia. Each sample contains a question, a gold answer (the reference answer supplied by the benchmark), a set of context documents drawn from Wikipedia, and a set of supporting facts that mark which sentences in the context are required to derive the answer. Questions are annotated with one of four types that describe the reasoning structure. These types are `comparison`, `compositional`, `bridge_comparison`, and `inference`.

2WikiMultihopQA is chosen because it is public, which makes the experiment reproducible, and because it is multi-hop by construction, so the answer cannot in general be read off a single passage. The multi-hop property is the setting in which graph-enhanced retrieval is expected to differ most from vector retrieval. The supporting-fact annotations also provide traceable evidence metadata, although this thesis uses them only for dataset traceability and does not perform a full supporting-fact analysis. Other multi-hop benchmarks such as HotpotQA and MuSiQue [9, 10] share the same motivation. Evaluating all three benchmarks jointly would have been preferable, but indexing and dual-mode execution are costly, so in the interest of compute and time a single benchmark was used. 2WikiMultihopQA was preferred because, in preliminary testing across the three benchmarks under LightRAG's dual-mode execution, it produced the most balanced `mix_required` versus `naive_enough` distribution, whereas the other two gave more skewed distributions that would have made supervised routing harder to evaluate; extending the study to them is left to future work.

3.1.2 Preprocessing and indexing

Each sample is converted into a structured record containing a stable sample identifier, the question, the gold answer, the question type, the context documents, and the supporting facts. The mapping between sample identifier and these fields is preserved throughout the pipeline so that every later artifact (retrieval output, judge label, router prediction) can be traced back to the original sample.

Context documents from these structured records are then inserted into LightRAG [3], which during indexing builds both a vector index over chunked text and an entity-relation graph derived from the same chunks. Once the index has been built, it is treated as a fixed part of the experimental setup. Stochasticity in graph construction (entity extraction, relation extraction, summarization) is therefore fixed in the index and does not introduce additional variance during the dual-mode retrieval experiments.

The experiments use the benchmark's train split. Its $167{,}454$ questions were converted into sample records, and context documents were indexed in the benchmark's original order until, under the available compute budget, $2{,}279$ questions had all of their context documents in the index. The first $2{,}000$ of these queryable questions, again in the benchmark's original order, constitute the query set used in the rest of the pipeline. No random sampling was involved at any stage, and the selection was fixed before any retrieval output or label existed; the identities of the judged queries and of the dataset splits are published with the code (Appendix A).

3.2 Routing-label pipeline

Figure 3.1 shows the end-to-end pipeline. Starting from 2WikiMultihopQA samples, the corpus is indexed in LightRAG, every sample is executed with both *Naive* and *Mix* retrieval, an LLM-as-a-Judge assigns a routing class to each query, and the resulting binary labels are used to train and evaluate query-only routers.

3.2.1 Dual-mode retrieval

For every benchmark question each of the $2{,}000$ selected questions q_i (Section 3.1.2), the indexed corpus is queried with both LightRAG retrieval modes, giving

$$r_i^N = R_N(q_i), \qquad r_i^M = R_M(q_i), \tag{3.1}$$

where R_N denotes LightRAG *Naive* (the low-cost vector path) and R_M denotes LightRAG *Mix* (the graph-enhanced path). The recorded artifacts for each query are the two generated answers r_i^N and r_i^M, the measured execution time t_i^N and t_i^M for each mode, and the number of context documents and supporting facts associated with the sample.

Two cost dimensions are reported. Execution time is logged per query for every query in the dataset. Model-token usage is additionally measured on an instrumented subset of $N = 150$ queries spanning $N = 15$ queries — the first 15 queries of the same processing stream, which span all four 2WikiMultihopQA question types although they were not sampled at random; their identities are published with the code. The reported token count per query is the total model-token usage returned by the Gemini API application programming interface (API) `usage_metadata` for each generation call, which comprises the prompt (input) tokens, the generated output tokens, and the model's internal reasoning ("thinking") tokens; the reasoning component is nonzero in every instrumented call. The embedding calls did not report token usage through the API, so their input sizes are estimated separately with the configured tokenizer; these estimates are on the order of tens of tokens per query and are excluded from the reported totals.

The instrumented subset is used to estimate the per-mode mean model-token consumption, denoted $\bar{u}^N$ for the *Naive* retrieval mode and $\bar{u}^M$ for the *Mix* retrieval mode. These per-mode means are then applied to the held-out test split for the routed-cost analysis in Section 3.5.2. The routed token figures are therefore estimated from per-mode token means and are not measured per query for every test example. Section 5.6.1 discusses the implications of this small instrumented subset. Reporting both cost dimensions matters because execution time is wall-clock time that includes the network round-trips of the for the API-based LightRAG and judge callsquery — retrieval, generation, and the associated network round-trips, not just model inference , — so it would shrink with a locally hosted model; model-token usage, by contrast, corresponds more directly to per-query operational cost in production deployments and is insensitive to network conditions.

Both LightRAG retrieval modes generate their answers with the same instruction-tuned Gemini model, and the LLM-as-a-Judge uses that same model; all model calls run at temperature 0 for reproducibility, and retrieval uses a Gemini embedding model over the chunked corpus. Using one capable but inexpensive model for both generation and judging keeps the pipeline simple and keeps the judge's reading of an answer consistent with how that answer was produced. The exact generator, judge, and embedding model checkpoints, the LightRAG version, retrieval parameters, token logging method, and judge prompt are listed in Appendix B.

3.2.2 LLM-as-a-Judge labeling

The 2WikiMultihopQA benchmark provides gold answers, but the routing labels needed here are specific to the two retrieval modes being compared and must therefore be constructed. They are constructed by running an LLM-as-a-Judge [18] that compares each query's two generated answers against the benchmark answer. The judge assigns one of three classes to each query. The class `naive_enough` is assigned when r_i^N matches the gold answer, so the low-cost mode was judged sufficient. The class `mix_required` is assigned when r_i^N does not match the gold answer but r_i^M does, so *Mix* was judged beneficial relative to *Naive* under the experimental pipeline. The class `none_enough` is assigned when neither mode produced an answer that matches the gold answer.

The binary routing label $y_i \in \{0, 1\}$ is then defined as

$$y_i = \begin{cases} 0, & \text{judge class is } \texttt{naive\_enough}, \\ 1, & \text{judge class is } \texttt{mix\_required}, \end{cases} \tag{3.2}$$

with `none_enough` cases excluded from the binary router training set. For `none_enough` queries neither retrieval mode succeeds, so the failure is one of answer quality and lies outside the scope of routing.

The label `mix_required` is operational: it records that *Mix* was judged beneficial relative to *Naive* under this LightRAG setup, without identifying which component of *Mix* (graph traversal, context assembly, or retrieval breadth) drove the improvement.

The LLM-as-a-Judge is treated as judged supervision and is not used as ground truth. Three Two design choices reduce the risk of judge error. The judge sees the gold answer in addition to the two candidate answers, which turns the task into a constrained comparison against a reference instead of open-ended quality scoring and makes the judgment easier and more reliable [18]. The same judge model and prompt are used across the entire dataset, which keeps the labeling protocol internally consistent.

3.3 Binary routing dataset

Before the binary dataset is analyzed by split, Table 3.1 summarizes how the raw dual-mode runs are filtered into the final binary routing dataset.

After the dual-mode execution and judge labeling, the binary routing dataset contains 1 286 judged queries with a routing class of either

Table 3.1: Label-flow from raw benchmark samples to the final binary routing dataset, showing how many queries pass each pipeline stage. The indented rows split the 2 000 judged queries into the three judge classes; only `naive_enough` and `mix_required` enter the binary dataset, while `none_enough` (neither mode correct) is excluded.

Stage	**Count**
Queries successfully run with both *Naive* and *Mix*	2 000
Judge produced usable label	2 000
`naive_enough`	959
`mix_required` (*Mix*-beneficial)	327
`none_enough` (excluded from binary dataset)	714
Final binary routing dataset	1 286

`naive_enough` or `mix_required`. Of these, 959 (74.6 %) are labeled `naive_enough` and 327 (25.4 %) are labeled `mix_required`. The class distribution is therefore imbalanced, which is taken into account during training and during reporting (Section 3.5).

The dataset is split into 900 training examples, 193 validation examples, and 193 test examples (approximately 70/15/15), jointly stratified by question type and routing label so that both the class proportions and the per-question-type proportions appear in all three splits in the same proportion as in the overall dataset. The validation split is held out from training and is used for early stopping and threshold selection. The test split is held out from both training and threshold selection and is used for final reported metrics.

Table 3.2 summarizes the routing dataset by split and 2WikiMultihopQA question type. The class proportions are preserved across splits by construction, but the distribution of `mix_required` labels is uneven across question types. `Compositional` questions are the most likely to be judged *Mix*-beneficial, while `comparison` questions are usually answered well enough by the low-cost mode.

3.4 Router models

Both router models take only the question text q_i as input and output a score $s_\phi(q_i) \in [0, 1]$, where ϕ denotes the model parameters. The score is interpreted as $P_\phi(y_i = 1 \mid q_i)$, the model's estimated probability that the binary routing label y_i is 1, i.e., that *Mix* is judged beneficial for query q_i under

Table 3.2: Routing-dataset composition by split and 2WikiMultihopQA question type (the four types are defined in Section 3.1.1). Each per-type cell gives the number of examples of that type in the split, with the number labeled `mix_required` in parentheses. Splits come from the joint stratified train/validation/test partition.

Split	**Total**	**comparison**	**compositional**	**bridge comp.**	**inference**
Train	900	415 (8 mix)	305 (178 mix)	155 (37 mix)	25 (6 mix)
Validation	193	89 (2 mix)	66 (38 mix)	33 (8 mix)	5 (1 mix)
Test	193	90 (2 mix)	65 (38 mix)	33 (8 mix)	5 (1 mix)
Overall	1 286	594 (12 mix)	436 (254 mix)	221 (53 mix)	35 (8 mix)

the experimental pipeline.

Both routers are trained on the 900-example training split of the binary routing dataset described in Section 3.3. The validation split is used only for early stopping (ModernBERT) and threshold selection (both routers). The test split is used only for the metrics reported in Chapter 4.

3.4.1 TF-IDF logistic-regression baseline

The baseline router is a logistic regression trained on TF-IDF features of the question. The pipeline first builds a vocabulary of V terms from the training questions, where each term is a unigram or bigram after lowercasing and basic English stop-word removal (scikit-learn `ngram_range` set to `(1, 2)`, a minimum document frequency of 2, and a vocabulary cap of $20{,}000$ features). Each question is then represented as a sparse vector $\mathbf{x}_i \in \mathbb{R}^V$ whose entries are the term frequency of each vocabulary term multiplied by the inverse document frequency of that term in the training corpus. The logistic-regression head produces the router score as

$$s_\phi(q_i) = \sigma(\mathbf{w}^\top \mathbf{x}_i + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{3.3}$$

where the weight vector $\mathbf{w} \in \mathbb{R}^V$ and bias $b \in \mathbb{R}$ are learned by minimizing the L_2-regularized binary cross-entropy loss with balanced class weights, so that the minority `mix_required` class is not ignored.

The baseline is intentionally simple. It has no pretrained semantic representations and can only exploit lexical patterns in the question. Its role is to indicate how much of the routing signal is captured by surface lexical cues, which calibrates how much improvement a transformer encoder can plausibly add.

The baseline is built with scikit-learn [19]. A logistic regression with L_2 regularization (regularization strength $C = 1.0$) is fitted on top of these features, with class weights set to `balanced` so that the loss on each example is inversely proportional to the frequency of its class in the training data. Optimization uses scikit-learn's limited-memory BFGS (L-BFGS) solver [20] with a maximum of $1{,}000$ iterations. Fitting on the 900 training questions takes a few seconds on a single central processing unit (CPU). The same fitted pipeline (vectorizer plus regression head) is then used to score the validation and test splits, producing a probability that each query is *Mix*-beneficial under the judged setup.

3.4.2 ModernBERT router

The transformer router is a ModernBERT-base [16] encoder with a binary classification head. The question is tokenized and encoded by ModernBERT into a sequence of contextualized hidden states. The pooled representation $h_i \in \mathbb{R}^d$, where d is the ModernBERT hidden dimension, is then mapped to two logits by the classification head,

$$z_i = Wh_i + b, \qquad W \in \mathbb{R}^{2\times d}, \quad b \in \mathbb{R}^2, \tag{3.4}$$

where logit $z_{i,0}$ corresponds to `naive_enough` and $z_{i,1}$ to `mix_required`. The router score is the softmax probability of the `mix_required` class,

$$s_\phi(q_i) = \frac{e^{z_{i,1}}}{e^{z_{i,0}} + e^{z_{i,1}}}. \tag{3.5}$$

Equivalently, the same probability can be written using a single logit and the sigmoid function, $s_\phi(q_i) = \sigma(z_{i,1} - z_{i,0})$. The two-logit parameterization used here matches the default Hugging Face classification-head format for binary classification with `num_labels=2`.

The model is fine-tuned by minimizing class-balanced cross-entropy on the n training examples,

$$\mathcal{L}(\phi) = -\frac{1}{n}\sum_{i=1}^{n} \alpha_{y_i} \log P_\phi(y_i \mid q_i), \tag{3.6}$$

where $\alpha_c = n/(2\,n_c)$ for class $c \in \{0, 1\}$, with n_c the number of training examples of class c. This weighting balances the contribution of the two classes so that the minority `mix_required` class is not down-weighted by the imbalanced training distribution. Optimization uses AdamW

The ModernBERT router uses the `answerdotai/ModernBERT-base` checkpoint from Hugging Face [16] with a binary classification head. Questions are tokenized with the standard ModernBERT tokenizer and truncated or padded to a maximum length of 128 tokens, which accommodates every question in the 2WikiMultihopQA splits.

Training uses the AdamW optimizer [21] with a learning rate of 2×10^{-5}, a weight decay of 0.01, gradient accumulation of two steps with a per-device batch size of 8 (effective batch size 16), a linear warm-up and linear decayschedule. The best checkpoint is selected on the validation split using the over the first $10\,\%$ of steps followed by linear decay, and gradient clipping at 1.0. The model is trained for up to 8 epochs with early stopping on validation F1 score on the `mix_required` class , and that checkpoint is then (at least 3 epochs before stopping, patience 3 epochs), and the checkpoint with the best validation F1 is used for all reported test metrics. Hyperparameters are listed

To distinguish variation due to random initialization from uncertainty due to the finite test set, the ModernBERT router is trained with three different random seeds (7, 13, 42). Each seed independently selects its own best checkpoint on the validation split and its own decision threshold on the validation split. The three runs are combined into a single seed-averaged ensemble (averaging the per-query *Mix* scores), which is the ModernBERT router reported in Chapter 4. 4; its decision threshold is selected on the averaged validation scores. All ModernBERT training and inference was performed on CPU. The model and dataset are small enough that the full fine-tuning run completes in a few hours on a single CPU, which is part of why a lightweight encoder router is attractive as a routing layer.

3.5 Evaluation metrics and statistical testing

Evaluation separates routing-label prediction from routed-cost analysis. Held-out classification metrics measure agreement with the judge-derived routing label. Routing a `naive_enough` query to *Mix* counts as a routing-label error because the lower-cost mode was already sufficient, while routing a `mix_required` query to *Naive* counts both as a routing-label error and as an answer-quality risk under the label definition.

Offline policy evaluation measures what happens when the router's predictions are used to select between the precomputed *Naive* and *Mix* retrieval outputs. Both evaluations use the held-out test split. The validation split is used only for early stopping and for selecting an operating threshold. Section 3.5.3 describes the statistical tests used to compare the routers on the shared test set.

3.5.1 Held-out classification metrics

Because the binary class distribution is imbalanced, the primary threshold-independent metric is the AUPRC on the mix_required class. AUPRC measures how well the router ranks mix_required queries above naive_enough queries and is more informative than area under the receiver operating characteristic curve (AUROC) when the positive class is in the minority [22]. AUROC is also reported for comparison with prior work.

For thresholded evaluation, the router score $s_\phi(q_i)$ is converted to a class prediction by applying a decision threshold $\tau \in [0, 1]$. The resulting routing policy π_τ maps each query to one of the two retrieval modes,

$$\pi_\tau(q) = \begin{cases} M, & s_\phi(q) \geq \tau, \\ N, & s_\phi(q) < \tau, \end{cases} \tag{3.7}$$

where M denotes *Mix* and N denotes *Naive*. Routing is treated as a binary classification problem with mix_required as the positive class and naive_enough as the negative class, and the metrics reported below are the standard binary-classification metrics for that labeling. At a chosen threshold these are routing-label accuracy, balanced routing-label accuracy, precision P and recall R on the positive (mix_required) class, and the F1 score

$$F_1 = \frac{2PR}{P + R}. \tag{3.8}$$

Throughout this thesis the word accuracy is used in the routing-label sense. It measures whether the policy selected the same mode as the judge-derived binary label. Balanced routing-label accuracy is included because the class distribution is imbalanced and plain routing-label accuracy can be high even for a trivial always-*Naive* policy.

Where bracketed confidence intervals are reported, they are computed using a 1 000-iteration bootstrap over the held-out test set. Each bootstrap sample resamples the test rows with replacement and recomputes the corresponding metric, and reported intervals are the 2.5th and 97.5th percentiles of the resulting bootstrap distribution.

ModernBERT is trained with three random seeds, and the reported ModernBERT router is a The

ModernBERT router reported in Chapter 4 is the seed-averaged ensemble of the three runs, formed by averaging the per-query *Mix* scores. The three individual runs vary little on the aggregate routing-label metrics (standard deviations below 0.02 across seeds), so the ensemble does not mask instability across seeds. The ensemble is then Section 3.4.2. It is bootstrapped on the test set exactly as the TF-IDF baseline is, so both routers carry comparable 95 % bootstrap confidence intervals rather than different dispersion measures; the three individual training runs are reported separately in Section 4.6.

3.5.2 Offline routing policy

The offline policy evaluation reuses the precomputed retrieval outputs and execution times from Section 3.2.1. For each held-out query, both r_i^N and r_i^M are already available, along with their execution times t_i^N and t_i^M. A routing policy π_τ at threshold τ is then simulated row by row. For each query the policy selects either the *Naive* or the *Mix* precomputed result, and aggregate statistics are computed across the entire held-out split.

Let m denote the number of held-out queries, and let $\mathbf{1}\{\cdot\}$ denote the indicator function that returns 1 when its condition holds and 0 otherwise. Let $r_M(\pi_\tau) = \frac{1}{m}\sum_{i=1}^{m}\mathbf{1}\{\pi_\tau(q_i) = M\}$ denote the fraction of test queries routed to *Mix* under policy π_τ. Two cost metrics are reported. The mean routed execution time $C(\pi_\tau)$ averages the per-query execution time of whichever retrieval mode the policy selected,

$$C(\pi_\tau) = \frac{1}{m}\sum_{i=1}^{m}\left[\mathbf{1}\{\pi_\tau(q_i) = N\}\, t_i^N + \mathbf{1}\{\pi_\tau(q_i) = M\}\, t_i^M\right]. \tag{3.9}$$

The mean routed model-token cost $U(\pi_\tau)$ is computed in the same way as the mean routed time in Equation 3.9, but with the per-mode token means $\bar{u}^N$ and $\bar{u}^M$ from the instrumented subset (Section 3.2.1) in place of the per-query execution times. The mean routed token cost is therefore an extrapolation from the instrumented subset and is reported alongside the per-query execution time in Chapter 44.

The same simulation is run for two static baseline policies, always-*Naive* and always-*Mix*, which bracket what a static retrieval strategy can achieve on the test set without any per-query decision. They serve as the reference points against which the learned router's routing-label and routed-cost trade-off is reported.

In addition to mean routed cost, the offline policy evaluation reports two routing error rates with different practical meanings. The under-routing rate, which is the false-negative rate of the binary classifiercounts the binary classifier's false negatives as a fraction of all test queries, is the fraction of test queries where the true label is `mix_required` but the policy routes to *Naive*,

$$\text{under-routing}(\pi_\tau) = \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\{y_i = 1,\, \pi_\tau(q_i) = N\}, \tag{3.10}$$

and the over-routing rate, the false-positive ratewhich counts its false positives in the same way, is the fraction of test queries where the true label is `naive_enough` but the policy routes to *Mix*,

$$\text{over-routing}(\pi_\tau) = \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\{y_i = 0,\, \pi_\tau(q_i) = M\}. \tag{3.11}$$

The two error types carry different practical weight. Under-routing is the error type most directly associated with answer-quality risk, because the policy selected *Naive* on a query where *Mix* was judged necessary. Over-routing is primarily a cost error, because the policy selected *Mix* on a query where *Naive* was already judged sufficient. Reporting them separately matters because the headline routing-label metrics count both as errors of the same kind.

Threshold selection uses the validation split only. The threshold reported on test is the one that maximizes F1 on the `mix_required` class on the validation split, swept over a fixed grid from 0.05 to 0.95 in steps of 0.01, and the same threshold is then applied unchanged to test. This prevents the threshold from being tuned on the data it is evaluated on.

3.5.3 Statistical testing

This chapter describes the implementation and the experiments. Section 3.4 describes how the two router models were trained. Section 3.6.1 presents the held-out classification experiment, which evaluates how well each router predicts the judge-derived routing label. Section 3.6.2 presents the offline routed-cost experiment, which evaluates each routeras a thresholded routing policy against the static always-*Naive* and always-*Mix* baselines. Section 4.1 reports the results of both experiments. Section 5 discusses what the results imply about the predictability of retrieval-mode selection, where the router fails, and what the practical routing-label and routed-cost trade-off looks likeThe TF-IDF baseline, the ModernBERT ensemble, and the type-only heuristic (Section 3.6.1) are all evaluated on the same 193 test queries. Differences between them are therefore assessed with paired procedures that condition on the shared test set [23]. The marginal bootstrap intervals described above quantify the uncertainty of each router's own metric, but the overlap of two marginal intervals is not a test of their difference and is not used as one. Three paired procedures are used, with every difference defined as ModernBERT minus TF-IDF (or learned router minus heuristic).

Both routers are trained on the 900-example training split of the binary routing dataset described in Section 3.3. The validation split is used only for early stopping (ModernBERT) and threshold selection (both routers). The test split is used only for the metrics reported in Section 4.1.

The baseline is built with scikit-learn [19]. Questions are lowercased and stripped of standard English stop words, then converted into sparse term-frequency vectors with unigram and bigram features (scikit-learn `ngram_range` set to `(1,2)`), a minimum document frequency of 2, and a vocabulary cap of 20,000 features. The term frequencies are weighted by inverse document frequency [17] learned on the training split. A logistic regression with L_2 regularization (regularization strength $C = 1.0$) is fitted on top of these features, with class weights set to `balanced` so that the loss on each example is inversely proportional to the frequency of its class in the training data. Optimization uses scikit-learn' s L-BFGS solver [20] with a maximum of 1,000 iterations. Fitting on the 900 training questions takes a few seconds on a single CPU. The same fitted pipeline (vectorizer plus regression head) is then used to score the validation and test splits, producing a probability that each query is *Mix*-beneficial under the judged setup.

The ModernBERT router uses the `answerdotai/ModernBERT-base` checkpoint from Hugging Face [16] with a binary classification head. Questions are tokenized with the standard ModernBERT tokenizer and truncated or padded to a maximum length of 128 tokens, which accommodates every question in the 2WikiMultihopQA splits.

Training uses the AdamW optimizer [21] with a learning rate of 2×10^{-5}, a weight decay of 0.01, gradient accumulation of two steps with a per-device batch size of 8 (effective batch size 16), a linear warm-up over the first $10\,\%$ of steps followed by linear decay, and gradient clipping at 1.0. The loss is class-balanced cross-entropy with weights α_c inversely proportional to class frequency in the training split, so that the minority

Paired bootstrap. The test rows are resampled with replacement $10\,000$ times; in each resample both routers are scored on the same rows, and the 2.5th and 97.5th percentiles of the resulting metric differences give a $95\,\%$ confidence interval for the difference [24]. A difference whose interval excludes zero is significant at the $5\,\%$ level; an interval whose endpoint lies exactly at zero is reported as not excluding zero. Thresholds are held fixed at their validation-selected values in every resample, so the intervals for the thresholded metrics do not include the variability of threshold selection (Section 4.6 shows its effect across seeds).

Paired permutation test. Under the null hypothesis that the two routers are exchangeable, which router produced a given query's output is arbitrary. In each of $10\,000$ permutations, the two routers' outputs are swapped on a random subset of test queries (each query independently with probability $1/2$) and the metric difference is recomputed. For the thresholded metrics the swapped outputs are the $0/1$ routing decisions at each router's own validation-selected threshold; for the ranking metrics (AUPRC and AUROC) they are each router's within-model score ranks, which leaves the metrics unchanged (both are invariant to within-model rank transformations of the scores) but prevents the routers' different score calibrations from confounding the test. The two-sided p-value is the fraction of permuted differences at least as large in magnitude as the observed difference, with the usual $+1$ correction in numerator and denominator. For AUROC the result is cross-checked with DeLong's test for correlated receiver operating characteristic curves [25].

McNemar's exact test. For the thresholded routing decision, each test query is either routed correctly or incorrectly by each router. McNemar's test [26] compares two routers on the discordant queries, those that exactly one of them routes correctly, with a two-sided exact binomial test. It is applied to all test rows (routing-label accuracy), to the `mix_required` class is not down-weighted by the imbalanced training distribution. The model is trained for up to 8 epochs with early stopping based on validation rows only (recall on the positive class, i.e. under-routing), and to the `naive_enough` rows only (over-routing), because the two error types carry different practical weight.

The significance level is $\alpha = 0.05$. Unadjusted p-values are reported for every comparison. For the seven metrics of the main comparison between the two learned routers (AUPRC, AUROC, balanced accuracy, precision, recall, and F1 on the `mix_required`class, requiring at least 3 epochs before stopping and using a patience of 3 epochs. `mix_required`, and routing-label accuracy), Holm-adjusted p-values [27] are reported alongside and are the basis for calling a difference significant; the five thresholded-metric comparisons of each learned router with the type-only heuristic form a second family that is Holm-adjusted in the same way, and the per-question-type comparisons are exploratory and are reported unadjusted. McNemar's test is used as a confirmatory cross-check of the permutation result on the corresponding metric (routing-label accuracy, recall, over-routing) rather than as an additional family of hypotheses, and its p-values are reported unadjusted. With 49 `mix_required` queries in the test split the tests have limited power to detect small differences, so a non-significant result is read as undecided rather than as evidence of no difference; Section 5.6.5 returns to this point. The paired procedures use a pseudo-random stream (seed

20260822) separate from the marginal bootstrap intervals, and all of them are implemented in the script listed in Appendix B.1.

To separate variance from random initialization from variance from finite test data, the ModernBERT router is trained with three different random seeds (7, 13, 42) . Each seed independently selects its own best checkpoint on the validation split and its own decision threshold on the validation split

3.6 Experimental design

Two main experiments and three secondary analyses are run on the held-out test split. All of them reuse the artifacts produced by the pipeline in Section 3.2; no retrieval is re-run at evaluation time. Section 3.6.1 describes the held-out classification experiment, which asks how well each query-only router predicts the judge-derived routing label and addresses the first sub-question of Section 1.2.2. Section 3.6.2 describes the offline routed-cost experiment, which asks what each router's decisions cost relative to the static baselines and addresses the second sub-question. Section 3.6.3 describes the secondary analyses: a per-question-type breakdown, an estimate of the judged answer quality each policy retains, and a per-seed stability check of the ModernBERT router. The three runs are combined into a single seed-averaged ensemble (averaging the per-query *Mix* scores) , which is the ModernBERT router reported in Section 4.1; its decision threshold is selected on the averaged validation scores. All ModernBERT training and inference was performed on CPU. The model and dataset are small enough that the full fine-tuning run completes in a few hours on a single CPU, which is part of why a lightweight encoder routeris attractive as a routing layer.

3.6.1 Held-out classification experiment

The held-out classification experiment evaluates each router as a binary classifier of judge-derived routing labels on the test split. The router scores are first evaluated threshold-independently, by computing AUPRC on the `mix_required` class and AUROC . AUPRC is the primary metric because it is sensitive to performance on the minority class, which is the class that matters for the routing decision [22](Section 3.5.1), which measures how well each router ranks the queries that need *Mix* above those that do not, independently of any operating point.

For thresholded metrics, the decision threshold is selected on the validation split. The selection rule is the threshold $\tau^\star$ that maximizes F1 on `mix_required` on the validation split, swept over a fixed grid in $[0.05, 0.95]$. The selected $\tau^\star$ is then $\tau^\star$ is the validation-selected threshold of Section 3.5.2, applied unchanged to the test split, where routing-label accuracy, balanced routing-label accuracy, precision, recall, and F1 on `mix_required` are reported. A trivial always-*Naive* predictor is included as a simple baseline because the class imbalance makes plain routing-label accuracy easy to inflate.

A non-deployable diagnostic baseline is also included to test whether the learned query-only router improves over a heuristic that routes by a coarse question-category label. The category label is taken directly from the benchmark's per-question type annotation; it is never visible to the learned routers, which see only the query text. The rule predicts *Mix* for `compositional` questions and *Naive* for all other question types, because `compositional` is the only question type whose *Mix*-beneficial rate exceeds 50 % in the training split. This baseline is not a realistic deployment policy because such a category label is unavailable for arbitrary user queries; it is included only as a reference for how much of the routing signal a coarse categorical rule can already deliver, and as a way to measure what the learned query-only routers add on top of it.

For TF-IDF, bracketed values are Both learned routers are reported with the 95 % bootstrap intervals [24] over 1 000 test-set resamples. ModernBERT denotes the seed-averaged ensemble of the three training runs and is bootstrapped identically to TF-IDFof Section 3.5.1. The ModernBERT ensemble is compared with the TF-IDF baseline, and both learned routers are compared with the type-only heuristic, using the paired tests of Section 3.5.3.

3.6.2 Offline routed-cost experiment

The offline routed-cost experiment evaluates each router as a thresholded routing policy along the two cost dimensions defined in Section 3.5.2, following the cost–quality routing-evaluation framing introduced by RouterBench [5]. The precomputed retrieval outputs (the *Naive* and *Mix* answers from Equation 3.1) and per-query execution times for both LightRAG modes are reused, so that the simulation does not have to re-run any retrieval. A routing policy π_τ at threshold τ sends each test query either to the *Naive* or to the *Mix* precomputed result. The ; each policy's mean routed execution time is computed as in (Equation 3.9. The), extrapolated mean routed model-token costis computed the same way but with per-mode mean token usage from the instrumented subset described in Section 3.2.1 in place of per-query times, giving an extrapolated mean token cost. Under-routing and , and under- and over-routing rates are computed for each routing policy following (Equations 3.10 and 3.11.

For each router ,) are computed at its validation-selected threshold and compared with the always-*Naive* and always-*Mix* reference points. In addition, for each router the threshold curve is computed over τ values stepping from 0 to 1 in increments of 0.01, producing a trace of mean routed time and F1 on mix_required at each threshold. The same simulation is run for the two static baseline policies, always-, from which the threshold-sensitivity plots and the cost–performance frontier are drawn. The difference between the two learned policies' mean routed execution times is tested with the paired procedures of Section 3.5.3 applied to the per-query routed-time differences (bootstrap of the mean difference and a sign-flip permutation test); the routed token cost, being an extrapolation from per-mode means, carries no uncertainty interval or test.

3.6.3 Per-question-type analysis, answer-quality retention, and seed stability

Three secondary analyses complement the two main experiments. First, the classification metrics are recomputed on each of the four 2WikiMultihopQA question-type slices of the test split, at the same validation-selected thresholds, in order to locate where the routing signal comes from; the per-slice differences between the two learned routers are tested with the paired permutation test restricted to the slice. Because the benchmark's question-type label is not available for arbitrary user queries, this breakdown is a diagnostic device and not a deployment recipe.

Second, because the offline evaluation does not re-judge the routed answers, the judged answer quality of each routing policy is estimated from the labels under one explicit assumption: that *Mix* answers correctly every query on which *Naive* and was judged sufficient, so that the only answer-quality regressions a policy can cause are mix_required queries routed to *Naive*. Under this assumption always-*Mix* . These two points bracket what a static policy can achieve on the test set and serve as the reference points the learned routers are compared against. answers all 193 binary-dataset test queries correctly by construction, always-*Naive* answers the $74.6\,\%$ labeled naive_enough, and the fraction of test queries answered correctly by a policy π_τ equals $1 - \mathrm{under\text{-}routing}(\pi_\tau)$; the ratio of this fraction to always-*Mix*'s is reported as the policy's retained share of judged-correct answers. Section 5.6.3 discusses what this estimate does and does not capture.

Third, the three individual ModernBERT training runs are reported next to the ensemble, each at its own validation-selected threshold, to show how much of the ensemble's behavior depends on the random seed.

Tables 4.1 and 4.3 summarize the test-set results :

Chapter 4

Results and analysis

This chapter reports and analyses the results of the experiments described in Section 3.6. It is organized by experiment, and each section presents the corresponding result, explains it, and interprets it before moving on; the broader implications are taken up in Chapter 5. Section 4.1 reports the held-out classification experiment, whose purpose is to establish how well each query-only router predicts the judge-derived routing label and whether the differences between the routers are statistically supported. Section 4.2 reports the offline routed-cost experiment, whose purpose is to quantify what each router's routing decisions cost in execution time and model tokens relative to the static baselines, and to separate the two routing-error types. Section 4.3 examines how sensitive these results are to the decision threshold and presents the resulting cost–performance frontier. Section 4.4 breaks the classification results down by 2WikiMultihopQA question type to locate where the routing signal comes from. Section 4.5 estimates how much judged answer quality each routing policy retains relative to always-*Mix*. Section 4.6 reports the per-seed results behind the ModernBERT ensemble.

Unless stated otherwise, all numbers are computed on the 193-query test split (49 `mix_required`, 144 `naive_enough`) at the validation-selected thresholds, $\tau^\star = 0.48$ for TF-IDF and $\tau^\star = 0.44$ for the ModernBERT ensemble; bracketed values are $95\,\%$ bootstrap intervals; and p-values come from the paired tests of Section 3.5.3.

4.1 Routing-label classification performance

Table 4.1 reports the routing-label classification performancemetrics of every policy on the held-out test split, and Table 4.3 reports offline routed cost and policy diagnostics. The asymmetry between the two error types is discussed in Section 5. 4.2 reports the uncertainty of the two learned routers' metrics together with the paired comparison between them. Both learned routers separate the two classes far better than chance: their AUPRC of 0.742 (TF-IDF) and 0.769 (ModernBERT) is about three times the `mix_required` base rate of 0.254 (marginal bootstrap lower bounds 0.610 and 0.645), and their AUROC of 0.892 and 0.910 is well above 0.500. The judged retrieval-mode need is therefore partly predictable from the query text alone, which is the premise on which the rest of the analysis rests.

Table 4.1: Routing-label classification performance on the held-out test split (point estimates). All accuracy values are routing-label accuracies (agreement with the judge-derived label); *Bal. acc.* is their class-balanced version and P_{mix}, R_{mix}, $F1_{mix}$ are on the `mix_required` class at the validation-selected threshold. ModernBERT is the seed-averaged ensemble of the three training runs. The constant-score baselines have AUROC $= 0.500$ and AUPRC equal to the base rate 0.254; the type-only heuristic is non-deployable because it uses the benchmark's question-category label; AUPRC and AUROC are not reported for it (—) because a hard decision rule does not produce a continuous ranking score. Confidence intervals and paired tests are given in Table 4.2.

Policy	**AUPRC**	**AUROC**	**Bal. acc.**	$\mathbf{P_{mix}}$	$\mathbf{R_{mix}}$	$\mathbf{F1_{mix}}$	**Acc.**
Always-*Naive*	0.254	0.500	0.500	0.000	0.000	0.000	0.746
Always-*Mix*	0.254	0.500	0.500	0.254	1.000	0.405	0.254
Type-only diagnostic	—	—	0.794	0.585	0.776	0.667	0.803
TF-IDF	0.742	0.892	0.784	0.559	0.776	0.650	0.788
ModernBERT achieves higher point estimates than TF-IDF	0.769	0.910	0.802	0.767	0.673	0.717	0.865

Figure 4.1 shows the test-set precision–recall curves on the `mix_required` class for the TF-IDF baseline and the ModernBERT ensemble. The base rate (the test-set fraction of `mix_required`) is shown as a horizontal dashed line. The ModernBERT curve sits above the TF-IDF curve over most of the recall range, which matches the higher AUPRC point estimate in Table 4.1, although the AUPRC difference is not significant (Table 4.2). The AUROC curves in Figure 4.2 are closer

Table 4.2: Uncertainty and paired comparison of the two learned routers on the held-out test split ($n = 193$). For each metric the table gives the TF-IDF and ModernBERT (seed-averaged ensemble) values with $95\,\%$ bootstrap intervals ($1\,000$ resamples), the difference ModernBERT − TF-IDF with its $95\,\%$ paired-bootstrap interval ($10\,000$ resamples), and the two-sided p-value of the paired permutation test ($10\,000$ permutations), unadjusted and Holm-adjusted over the seven metrics. Differences that are significant at $\alpha = 0.05$ after adjustment are marked with *.

Metric	**TF-IDF**	**ModernBERT**	**Difference [95 % interval]**	p **(Holm)**
AUPRC	0.742 [0.610, 0.853]	0.769 [0.645, 0.889]	+0.027 [−0.053, 0.105]	0.52 (1.00)
AUROC	0.892 [0.836, 0.936]	0.910 [0.865, 0.949]	+0.018 [−0.009, 0.047]	0.165 (0.56)
Bal. acc.	0.784 [0.718, 0.848]	0.802 [0.730, 0.872]	+0.018 [−0.043, 0.075]	0.52 (1.00)
P_{mix}	0.559 [0.437, 0.681]	0.767 [0.644, 0.889]	+0.209* [0.115, 0.312]	< 0.001 (< 0.001)
R_{mix}	0.776 [0.650, 0.885]	0.673 [0.540, 0.809]	−0.102 [−0.212, 0.000]	0.127 (0.56)
$F1_{mix}$	0.650 [0.547, 0.740]	0.717 [0.607, 0.810]	+0.068 [−0.022, 0.155]	0.113 (0.56)
Acc.	0.788 [0.731, 0.845]	0.865 [0.819, 0.912]	+0.078* [0.026, 0.130]	0.006 (0.035)

together because AUROC is less sensitive to performance on the minority class.

ModernBERT has the higher point estimate on every aggregate routing-label metric: AUPRC , but the paired tests in Table 4.2 show that only part of this advantage is statistically supported on a 193-query test set. The differences in AUPRC ($+0.027$, $95\,\%$ paired-bootstrap interval $[-0.053, 0.105]$, permutation $p = 0.52$), AUROC ($+0.018$, $[-0.009, 0.047]$, $p = 0.165$; DeLong $p = 0.179$), balanced routing-label accuracy ($+0.018$, $[-0.043, 0.075]$, $p = 0.52$), and F1 on the *Mix*-beneficial class, and `mix_required` ($+0.068$, $[-0.022, 0.155]$, $p = 0.113$) all have intervals that include zero and are not significant. The difference in routing-label accuracy. The bootstrap intervals of the two routers overlap on every threshold-independent metric (Table 4.1) accuracy ($+0.078$, so these gaps are not statistically separated on this 193-row test set; the routing-label-accuracy gap (0.865 $[0.026, 0.130]$, $p = 0.006$; Holm-adjusted $p = 0.035$) is significant, and McNemar's

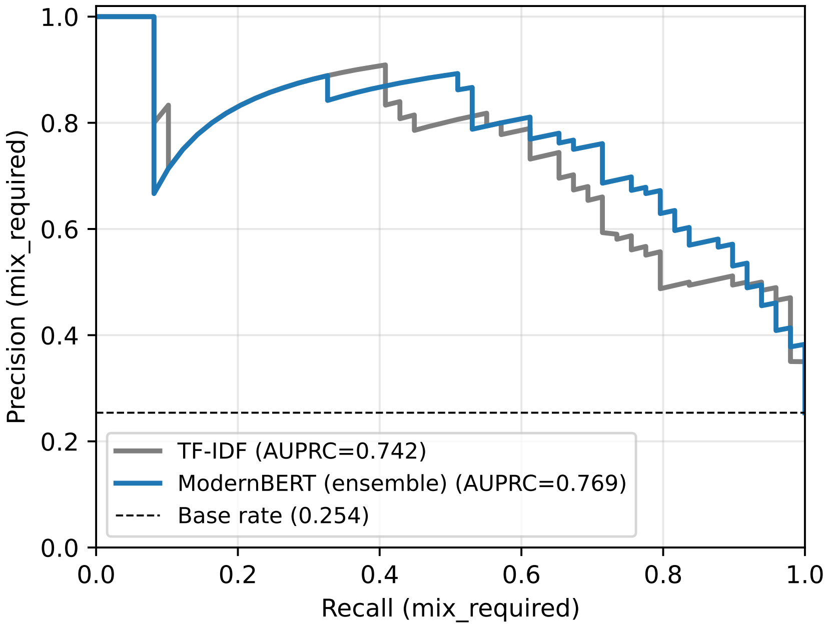

Figure 4.1: Precision–recall curves on the held-out test split for the TF-IDF baseline and the ModernBERT seed-averaged ensemble. The dashed horizontal line is the test-set base rate of `mix_required`.

test agrees: ModernBERT routes 21 test queries correctly that TF-IDF routes incorrectly, while the reverse holds for 6 ($p = 0.006$). The accuracy gain comes from precision, which is the largest and most clearly supported difference between the routers (0.767 vs. 0.788)is the widest and comes closest to separation. The gain comes mainly from higher precision. ModernBERT sends 0.559, $+0.209$, $[0.115, 0.312]$, $p < 0.001$, Holm-adjusted $p < 0.001$): ModernBERT sends far fewer *Naive*-sufficient queries to *Mix*, so its over-routing rate is lower (0.052 vs. 0.155 for TF-IDF). TF-IDF pulls in the other direction: it has higher *Mix* recall and a lower under-routing rate (0.057 . Its recall on `mix_required` is lower (0.673 vs. 0.083 for ModernBERT), so it misses fewer *Mix*-beneficial queries, at 0.776, -0.102, $[-0.212, 0.000]$, $p = 0.127$; the upper endpoint of the interval lies at zero, so the interval does not exclude zero), and this difference is not significant either: of the cost of routing more unnecessary queriesto *Mix*. 49 `mix_required` test queries, TF-IDF routes 6 correctly that ModernBERT misses and ModernBERT routes 1 correctly that TF-IDF misses (McNemar $p = 0.125$).

The two routers therefore sit at opposite ends of the precision-recall precision–recall trade-off on the minority class, and the statistically established part of the difference is on the precision side: ModernBERT is more selective and lower-cost;

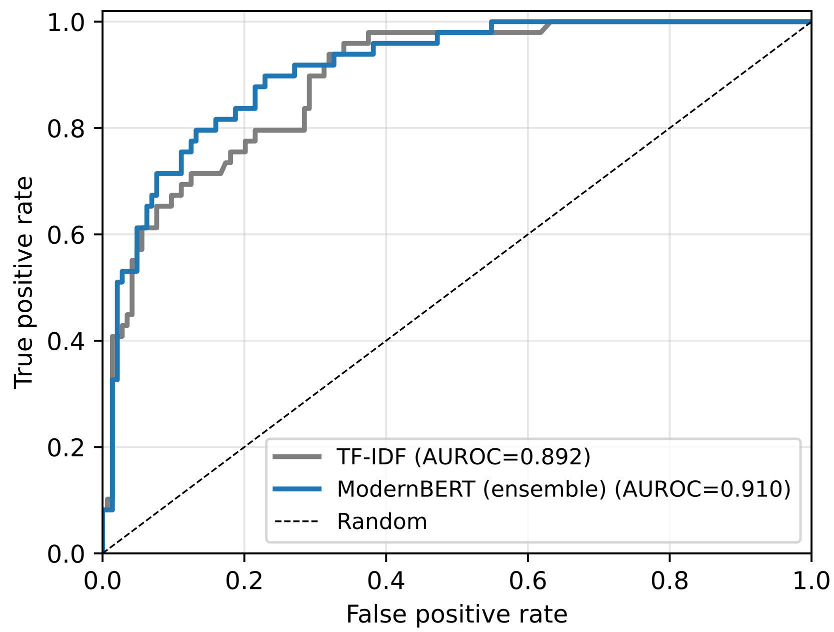

Figure 4.2: Receiver-operating-characteristic curves on the held-out test split.

TF-IDF is more conservative and spends more tokens the more selective router, and TF-IDF the more conservative one that spends more *Mix* calls to keep recall on the minority class up. up. Whether the encoder also *ranks* the queries better than the lexical baseline, as the higher AUPRC and AUROC point estimates suggest, cannot be decided on this test set: the paired-bootstrap intervals for the AUPRC, AUROC, and balanced-accuracy differences are roughly ± 0.03 to ± 0.08 wide, so a test set several times larger for AUROC and F1, and roughly an order of magnitude larger for AUPRC and balanced accuracy, would be needed to resolve gaps of the observed size (Section 5.6.5 quantifies this). The point estimates favor ModernBERT consistently, and Section 4.6 shows that they do so for every training seed, but a larger benchmark would be needed before the ranking-quality gap can be claimed. Section 4.2 shows what the established precision difference means in routed cost.

The type-only diagnostic heuristic is strong, which shows that a coarse question-category rule already captures a substantial part of the routing signal: predicting *Mix* for `compositional` questions and *Naive* otherwise reaches an F1 of 0.667 and a routing-label accuracy of 0.803 (Table 4.1), because `compositional` questions dominate the *Mix*-beneficial class on this benchmark. Its role is diagnostic only, because this heuristic needs a category label that the category label is unavailable for arbitrary user queries. ModernBERTachieves higher point estimates than the categorical heuristic on F1 (0.717 vs. 0.667), precision (, which sees only the query text and never this label, achieves significantly higher precision than the heuristic (0.767 vs. 0.585), and ; permutation $p < 0.001$, Holm-adjusted over the five heuristic comparisons $p = 0.002$) and higher routing-label accuracy (0.865 vs. 0.803); $p = 0.024$ unadjusted, Holm-adjusted $p = 0.096$; McNemar 19 vs. 7 discordant queries, $p = 0.029$), a difference that is significant before but not after the adjustment; its higher F1 (0.717 vs. 0.667, and it does so from query text alone.TF-IDF matches the heuristic 's *Mix* recall ($p = 0.24$), higher balanced accuracy ($p = 0.74$), and lower recall (0.673 vs. 0.776) but at lower precision and lower overall accuracy. , $p = 0.124$) are not significant. TF-IDF does not differ significantly from the heuristic on any metric (all $p > 0.56$; its recall of 0.776 equals the heuristic's exactly). The learned query-only router should therefore be read as predicting an operational label from phrasing: what it adds over the type-only reference reflects signal the encoder extracts from how questions are worded, and what it does not exceed reflects benchmark structure rather than a deeper property of the queries.

4.2 Routed cost and routing-error types

This experiment asks what each router's decisions cost relative to the static baselines, and which of the two routing-error types each router makes. Table 4.3 reports, for each policy, the fraction of test queries routed to *Mix*, the resulting mean routed execution time and estimated mean routed model-token usage, the token saving relative to always-*Mix*, and the two routing-error rates. The cost dimension follows the precision pattern of Section 4.1: TF-IDF routes 35.2 % of the test queries to *Mix* and ModernBERT 22.3 %, so TF-IDF routes at 8,531 mean tokens per query , and ModernBERT at 6,666, and against 17,857 for always-*Mix* at 17,857, which translates into and 3,458 for always-*Naive*. This corresponds to token savings of 52 % and 63 % relative to the expensive baseline. , and to mean routed execution times of 16.98 and 14.11 seconds per query against 24.42 seconds for always-*Mix*, i.e. 30 % and 42 % lower. The difference between the two learned policies' mean routed times (−2.87 seconds per query, 95 % paired-bootstrap interval [−5.57, −0.88], sign-flip permutation $p = 0.003$; the two policies route 27 of the 193 queries differently) is statistically significant; the token difference, by contrast, is an extrapolation from the instrumented subset and carries no interval.

Table 4.3: Routing-label classification performance Offline routed-cost and policy diagnostics on the held-out test split. All accuracy values are routing-label accuracies For each policy the table gives the fraction of queries routed to *Mix*, the resulting mean per-query time and estimated mean tokens (agreement with a mode-weighted average of the judge-derived labelper-mode means from the instrumented $N = 15$ subset); *Bal. acc.* is their class-balanced version and P_{mix}, $R_{mix}$the token saving relative to always-*Mix*, $F1_{mix}$ are on and the mix_required class at the validation-selected threshold. Bracketed values are 95 % bootstrap intervals under-routing (false negatives) and over-routing (false positives) fractions over 1 000 test-set resamplesall test queries. Under-routing carries the answer-quality risk; over-routing is mainly a cost error. ModernBERT is the seed-averaged ensembleof the three runs, bootstrapped identically to TF-IDF.The constant-score baselines have AUROC = 0.500 and AUPRC equal to the base rate 0.254; the type-only heuristic is non-deployable as it uses the benchmark's question-category label.

Offline routed-cost and policy diagnostics on the held-out test split. For each policy the table gives the fraction of queries routed to *Mix*, the resulting mean per-query time and estimated mean tokens (a mode-weighted average of the per-mode means from the instrumented $N = 150$ subset), the token saving relative to always-*Mix*, and the absolute false-negative (*Under-rout.*) and false-positive (*Over-rout.*) routing-error fractions over all test queries. Under-routing carries the answer-quality risk; over-routing is mainly a cost error. ModernBERT is the seed-averaged ensemble.

Policy	**Routed to *Mix***	**Mean time (s)**	**Mean tokens**	**Token saving**	**Under-routing**	**Over-routing**
Always-*Naive*	0.000	10.95	3 458	80.6 %	0.254	0.000
Always-*Mix*	1.000	24.42	17 857	0.0 %	0.000	0.746
Type-only diagnostic	0.337	15.59	8 308	53.5 %	0.057	0.140
TF-IDF	0.352	16.98	8 531	52.2 %	0.057	0.155
ModernBERT	0.223	14.11	6 666	62.7 %	0.083	0.052

Figure 4.1 shows the test-set precision–recall curves on Because under-routing carries the answer-quality risk and over-routing only wastes compute (Section 3.5.2), the two error types have to be read separately. The asymmetry explains why always-*Mix* scores poorly on routing-label accuracy without necessarily producing worse final answers, and it also explains the gap between TF-IDF and ModernBERT in Table 4.3: TF-IDF has lower under-routing (0.057 vs. 0.083), so it misses fewer *Mix*-beneficial queries; ModernBERT has lower over-routing (0.052 vs. 0.155), so it issues fewer unnecessary *Mix* calls. Of the two gaps, only the over-routing gap is statistically established: of the mix_required class for the TF-IDF baseline and the ModernBERT ensemble.The base rate (the test-set fraction of 144 mixnaive_required) is shown as a horizontal dashed line.The ModernBERT

curve sits above the TF-IDF curve over most of the recall range, which matches the higher AUPRC point estimate in Table 4.1, though the two AUPRC bootstrap intervals overlap. The AUROC curves in Figure 4.2 are closer together because AUROC is less sensitive to performance on the minority class. enough test queries, ModernBERT routes 20 correctly to *Naive* that TF-IDF sends to *Mix*, and there is no query on which the reverse happens (McNemar $p < 0.001$), whereas the under-routing gap rests on 6 versus 1 discordant queries ($p = 0.125$). ModernBERT's over-routing is also significantly lower than that of the type-only heuristic (18 vs. 1 discordant queries, $p < 0.001$), while neither learned router's under-routing differs significantly from the heuristic's ($p = 0.125$ for ModernBERT and $p = 1.00$ for TF-IDF).

Precision–recall curves on the held-out test split for the TF-IDF baseline and the ModernBERT seed-averaged ensemble. The dashed horizontal line is the test-set base rate of mix_required. Routing-label performance and routed cost must therefore be interpreted together. The selected ModernBERT operating point behaves selectively, saving more compute at the cost of accepting somewhat more under-routing risk. The TF-IDF operating point behaves conservatively in the opposite direction. Neither extreme is universally correct: which is preferable depends on the relative cost of an unnecessary *Mix* call against an answer-quality regression in a given deployment, and Section 4.3 shows that either router can be moved along this trade-off by changing its threshold.

Receiver-operating-characteristic curves on the held-out test split. The cost picture is sharper in model tokens than in execution time: the learned routers save 30–$42\,\%$ of always-*Mix*'s mean routed time but 52–$63\,\%$ of its estimated tokens. The token reduction is larger than the time reduction because *Mix* adds one extra LLM call per query and a substantially larger assembled context [3, 12], while execution time is partly absorbed by network latency that does not scale with token volume. The routing decision itself contributes zero LLM tokens because neither router calls a generative model; its inference cost (three encoder forward passes per query for the reported ensemble) was not included in the routed-time figures and is expected, though not separately measured, to be negligible relative to retrieval cost. The routed token figures are extrapolated from the per-mode means of the $N = 15$ instrumented subset (Section 3.2.1) and therefore carry no bootstrap interval; Section 5.6.1 discusses this limitation.

4.3 Threshold sensitivity and cost frontier

This section checks whether the operating points of Section 4.2 are brittle with respect to the validation-selected threshold, and places them on the cost–performance frontier traced out by sweeping the threshold. Figure 4.3 shows how F1 on `mix_required` varies with the decision threshold on the test split, with the validation-selected threshold of each model marked as a dot. The ModernBERT ensemble selects a validation threshold of $\tau^\star = 0.44$, and its test F1 is flat over a wide band of thresholds around this value, which indicates that the F1 plateau is broad and that the choice of operating threshold is not a brittle hyperparameter for these routers.

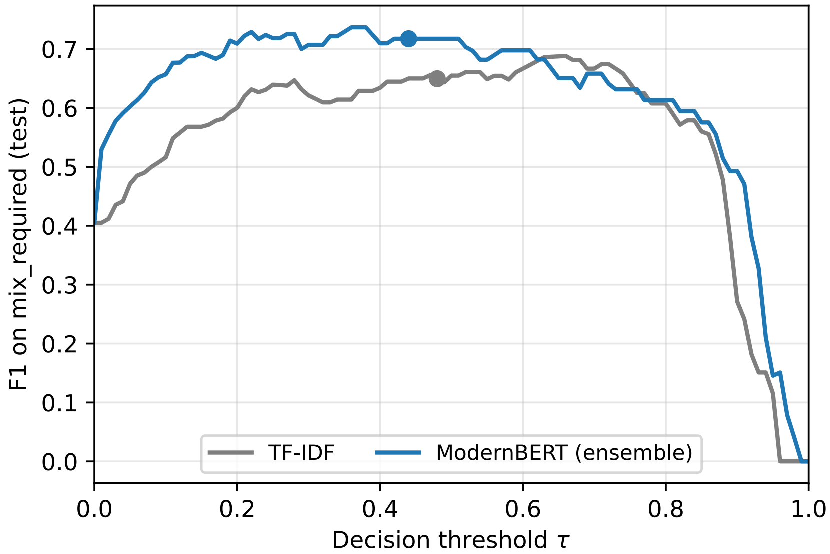

Figure 4.3: F1 on `mix_required` as a function of the decision threshold on the held-out test split. Dots mark the validation-selected threshold for each model: TF-IDF $\tau^\star = 0.48$ and the ModernBERT ensemble $\tau^\star = 0.44$.

Figure 4.4 shows the mean routed time on test as the threshold sweeps from 0 (every query routed to *Mix*) to 1 (every query routed to *Naive*). The always-*Naive* and always-*Mix* levels are shown as horizontal dashed lines and act as a sanity check. At $\tau = 0$ every router collapses to always-*Mix*, and at $\tau = 1$ every router collapses to always-*Naive*, which is what the figure shows.

Figure 4.5 combines the two preceding views into a frontier of routing-label F1 against mean routed execution time. Each router traces an operating curve in (mean routed time, $\mathrm{F1}_{\mathrm{mix}}$) space as τ sweeps. The static always-*Naive* and always-*Mix* policies appear as the two extreme operating points. Both learned routers achieve higher routing-label F1 than either static policy at substantially lower mean routed time than always-*Mix*. The ModernBERT

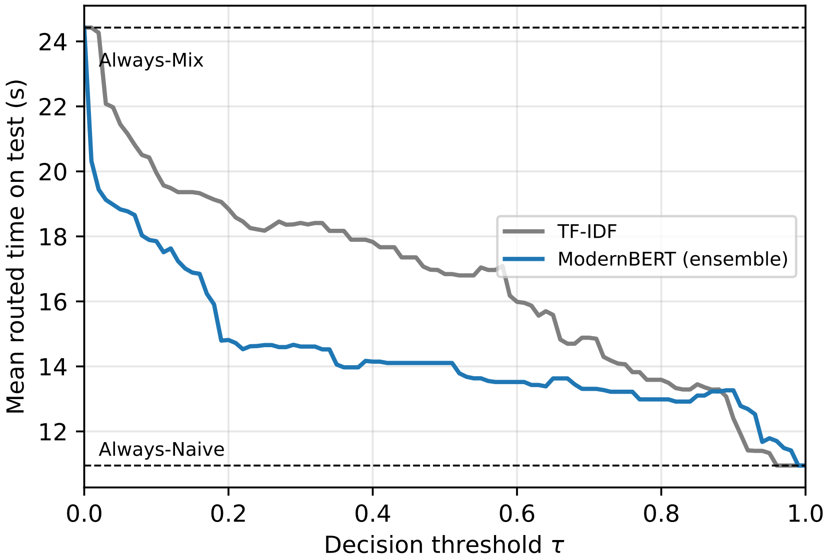

Figure 4.4: Mean routed execution time on the held-out test split as the decision threshold sweeps. The two horizontal dashed lines are the always-*Naive* and always-*Mix* reference levels.

operating point reaches a higher F1$_{\text{mix}}$ than TF-IDF lies at a lower mean routed time than TF-IDF's, a difference that is statistically significant (Section 4.2), and at a higher F1$_{\text{mix}}$ point estimate, although the F1 difference itself is not significant (Table 4.2).

The frontier shows that the two routers' threshold sweeps trace largely overlapping curves: the selected ModernBERT point reaches a higher F1$_{\text{mix}}$ $(0.717$ vs. $0.650)$ at 2.9 seconds lower mean routed time, and either router can be moved to the other's cost level by changing τ. The practical content of the frontier is therefore that the operating point, as much as the choice of router, controls where a deployment sits between always-*Naive* and always-*Mix*: the router determines how much routing-label F1 is available at a given cost, and the threshold selects the cost.

4.4 Per-question-type analysis

To locate where the routing signal comes from, the classification metrics are recomputed per 2WikiMultihopQA question type; these comparisons are exploratory and their p-values are unadjusted. Table 4.4 reports a per-question-type breakdown on the test split. The four 2WikiMultihopQA question types carry very different fractions of *Mix*-beneficial queries on this

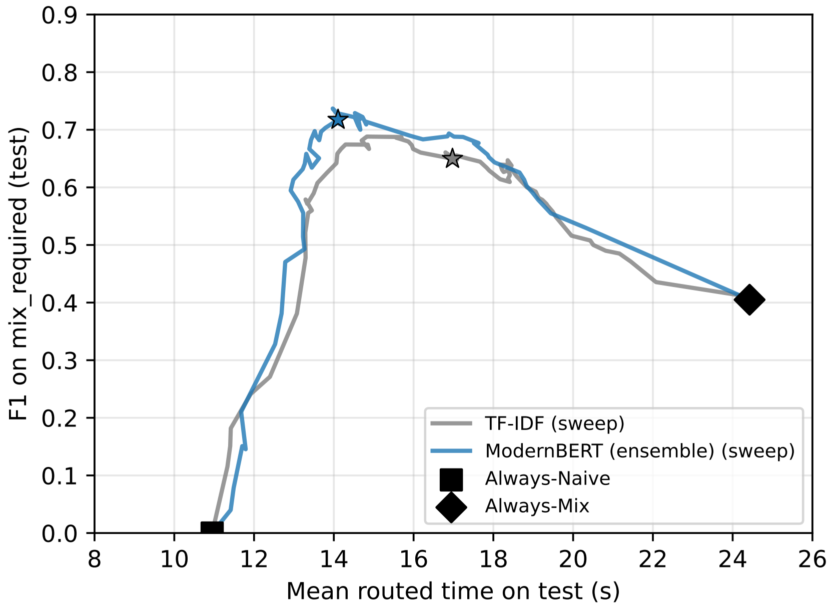

Figure 4.5: Routing-label F1 on the `mix_required` class versus mean routed execution time on the held-out test split. Each curve traces the operating points obtained by sweeping the decision threshold. Stars mark the validation-selected thresholds. Static always-*Naive* and always-*Mix* policies are shown as reference points.

split: 2 out of 90 for `comparison`, 8 out of 33 for `bridge_comparison`, 38 out of 65 for `compositional`, and 1 out of 5 for `inference`. The `compositional` slice is the one in which the routing decision matters most. The `inference` slice has too few rows to support stable estimates, so its per-class metrics are reported for completeness only. On `compositional`, ModernBERT achieves higher point estimates than TF-IDF on both AUPRC (0.834 vs. 0.814) and F1 on `mix_required` (0.810 vs. 0.753). On `bridge_comparison` the two routers trade strengths: TF-IDF routes more queries to *Mix* on this slice and reaches higher recall (0.375 vs. 0.125), slightly higher AUPRC (0.418 vs. 0.396), and higher F1 (0.316 vs. 0.200), while ModernBERT keeps higher precision (0.500 vs. 0.273) and higher overall slice accuracy (0.758 vs. 0.606). Of these per-slice differences, only ModernBERT's higher precision on `compositional` is significant (0.780 vs. 0.636, $+0.144$, paired-bootstrap interval $[0.058, 0.239]$, permutation $p = 0.003$), mirroring the aggregate precision result of Section 4.1; the others are not: on `compositional` the permutation p-values are 0.69 for AUPRC, 0.20 for F1, and 0.078 for slice accuracy, and on bridge_comparison they are 0.90, 0.51, and 0.180 respectively, so the remaining per-type comparisons describe where the two routers' point estimates differ without establishing that they do.

Table 4.4: Per-question-type breakdown on the held-out test split. Metrics are as a subset of those in Table 4.1 but (AUROC and balanced accuracy omitted), computed only on each slice's rows (threshold metrics at each router's validation-selected threshold); n n and n_{mix} n_{mix} are the slice size and its number of mix_required queries. The comparison and inference slices have too few positives (2 and 1) for stable AUPRC/F1 and are shown for completeness only. ModernBERT is the seed-averaged ensemble.

Slice	**Model**	**AUPRC**	**$\mathbf{P_{mix}}$**	**$\mathbf{R_{mix}}$**	**$\mathbf{F1_{mix}}$**	**Acc.**
comparison ($n = 90$, $n_{\text{mix}} = 2$)	TF-IDF	0.277	0.000	0.000	0.000	0.978
	ModernBERT	0.140	0.000	0.000	0.000	0.978
compositional ($n = 65$, $n_{\text{mix}} = 38$)	TF-IDF	0.814	0.636	0.921	0.753	0.646
	ModernBERT	0.834	0.780	0.842	0.810	0.769
bridge_comparison ($n = 33$, $n_{\text{mix}} = 8$)	TF-IDF	0.418	0.273	0.375	0.316	0.606
	ModernBERT	0.396	0.500	0.125	0.200	0.758
inference ($n = 5$, $n_{\text{mix}} = 1$)	TF-IDF	0.333	0.000	0.000	0.000	0.400
	ModernBERT	0.500	0.000	0.000	0.000	0.800

Figure 4.6 visualizes the AUPRC comparison on the two non-degenerate slices.

Beyond routing-label accuracy , it is useful to ask how much answer quality the cost saving costs The compositional slice is the most informative one because the class distribution is balanced and *Mix* is judged beneficial on most of its queries. The gain on this slice despite the small training set is consistent with the encoder capturing signal beyond the lexical patterns available to the TF-IDF baseline: its precision advantage is significant there, whereas the AUPRC and F1 gaps are not. The bridge_comparison slice is harder for both routers and is the natural target for follow-up work; on this slice TF-IDF reaches slightly higher AUPRC and F1 by routing more queries to *Mix*, while ModernBERT keeps higher precision and higher slice accuracy at the cost of missing more *Mix*-beneficial queries, and none of these differences is significant.

The per-question-type breakdown and the type-only diagnostic

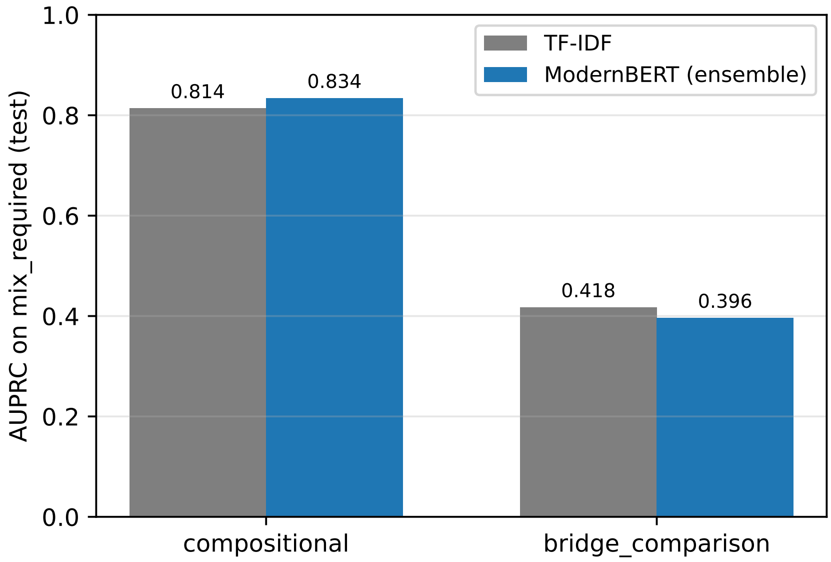

Figure 4.6: AUPRC on the `mix_required` class, broken down by 2WikiMultihopQA question type on the held-out test split. ModernBERT bars are the seed-averaged ensemble. The `comparison` and `inference` slices have too few `mix_required` test rows to give an informative AUPRC and are omitted.

baseline both rely on the benchmark's per-question category label. In a real deployment that label does not exist; users send free-form queries and the routing layer sees only the query text. The category-based slices are useful as a diagnostic device because they show which subpopulations the router handles well and which it does not, and they help separate signal that comes from coarse structural cues (such as whether the question is a compositional multi-hop lookup) from signal that comes from finer phrasing. They are not a deployment recipe; the learned routers approximate the same structure from the query text alone, which is the only input available outside the benchmark.

The same framing helps interpret the contrast between the `compositional` and `bridge_comparison` slices. The fact that ModernBERT is strong on `compositional` and weaker on `bridge_comparison` indicates which phrasings the encoder has learned to recognize on this benchmark, and does not extend to a claim that `compositional` queries are easier to route in production. Within this benchmark, the `compositional` slice contains more rows and the rows share a more consistent surface form.

4.5 Answer-quality retention

It is also useful to quantify the answer-quality loss associated with the cost saving. The judge labels imply, for each policy, how often the routed mode returns a correct answer (Section 3.6.3). Under the assumption that *Mix* answers correctly on every query where *Naive* already does — so that the only answer-quality regressions a policy can cause are *Mix*-beneficial queries routed to *Naive* — always-*Naive* answers $74.6\,\%$ of the test queries correctly and always-*Mix* answers all of them correctly by construction. The TF-IDF router retains $94.3\,\%$ and the ModernBERT ensemble $91.7\,\%$ of always-*Mix*'s correct answers, at $52\,\%$ and $63\,\%$ lower estimated token cost respectively. The ModernBERT ensemble thus trades roughly eight percentage points of answer recovery always-*Mix*'s judged-correct answers for a large cost reduction, and its answer-quality risk is concentrated entirely in its under-routing rate (0.083). The 2.6-percentage-point difference in retained answers between the two routers corresponds to the under-routing gap, which is not statistically significant (6 vs. 1 discordant queries, McNemar $p = 0.125$; Section 4.2), whereas their mean routed-time difference is; the token-cost difference remains an untested estimate from the instrumented subset. This estimate relies on the stated assumption about *Mix* correctness on `naive_enough` queries; re-judging the *Mix* answers on those queries would remove the assumption and is left to future work.

The retrieval-mode distinction between Always-*Mix* illustrates why Tables 4.1 and 4.3 must be read together. It is label-incorrect on *Naive*and *Mix* is partly predictable from the query text alone. Both learned routers outperform the always-*Naive* baseline on the minority class and reach operating points in Figure 4.5 that no static policy at comparable cost can match. The ModernBERT router achieves higher point estimates than TF-IDF on AUPRC, balanced routing-label -sufficient rows because it uses more retrieval than necessary, although the underlying *Mix* answer on those rows can still be correct. The absence of a direct final-answer-accuracy comparison for each routed policy is treated as a limitation in Section 5.6.3.

4.6 Seed stability of the ModernBERT router

To check that the ensemble's advantage is not an artifact of one seed, Table 4.5 reports the three individual ModernBERT training runs next to the ensemble. The aggregate metrics vary little across seeds (sample standard deviations of 0.011 to 0.017 for AUPRC, AUROC, balanced accuracy, F1 on the minority class, and routing-label accuracy, although the bootstrap intervals overlap on all of these except routing-label accuracy.

The two routing error types have different practical weight. A false negative routes a *Mix*-beneficial query to *Naive* and so risks an answer-quality regression on a query where *Mix* was judged necessary. A false positive routes a *Naive*-sufficient query to *Mix* and only wastes compute, because *Naive* was already judged sufficient. This asymmetry explains why always-*Mix* scores poorly on routing-label accuracy without necessarily producing worse final answers, and it also explains the gap between TF-IDF and ModernBERT in Table 4.3: TF-IDF has higher recall and lower under-routing, so it misses fewer *Mix*-beneficial queries; ModernBERT has higher precision and lower over-routing, so it issues fewer unnecessary *Mix* calls.

Routing-label performance and routed cost must therefore be interpreted together. The selected ModernBERT operating point behaves selectively, saving more compute at the cost of accepting somewhat more under-routing risk. The TF-IDF), and every individual seed has a higher point estimate than TF-IDF on all five of these metrics. Precision and recall vary more across seeds (0.057 and 0.042) because each run selects its own validation threshold (0.24 to 0.42), which moves the operating point behaves conservatively in the opposite direction. Neither extreme is universally correct: which is preferable depends on the relative cost of an unnecessary *Mix* call against an answer-quality regression in a given deploymentalong the precision–recall trade-off; the ensemble, whose averaged scores select a threshold of 0.44, sits at the high-precision end. The ensemble's advantage over TF-IDF is therefore not an artifact of one favorable seed, but the exact precision–recall balance of the reported operating point is threshold-dependent, as Figure 4.3 also shows.

Table 4.5: Test-split metrics of the three individual ModernBERT training runs, each at its own validation-selected threshold $\tau^{\star}$, their sample standard deviation across seeds ($n = 3$), and the seed-averaged ensemble reported elsewhere in this chapter. Metrics are as in Table 4.1.

Run	$\tau^{\star}$	**AUPRC**	**AUROC**	**Bal. acc.**	**P$_{\text{mix}}$**	**R$_{\text{mix}}$**	**F1$_{\text{mix}}$**	**Acc.**
Seed 7	0.36	0.754	0.895	0.791	0.648	0.714	0.680	0.829
Seed 13	0.42	0.783	0.915	0.792	0.762	0.653	0.703	0.860
Seed 42	0.24	0.771	0.915	0.812	0.692	0.735	0.713	0.850
Std. across seeds	—	0.015	0.011	0.012	0.057	0.042	0.017	0.016
Ensemble	0.44	0.769	0.910	0.802	0.767	0.673	0.717	0.865

Chapter 5

Discussion

This chapter discusses the results of Chapter 4. Section 5.1 summarizes the key findings. Section 5.2 relates them to prior work and states the research contribution. Section 5.3 discusses their industrial relevance and societal impact, Section 5.4 the ethical considerations, and Section 5.5 the sustainability aspects. Section 5.6 reflects on the limitations of the study, and Section 5.7 documents the use of generative AI tools during the project.

The compositional slice is the most informative one because the class distribution is balanced

5.1 Summary of key findings

The experiments support four findings. First, the retrieval-mode distinction between *Naive* and *Mix* is judged beneficial on most of its queries . On this slice ModernBERT achieves an AUPRC of 0.834 against 0.814 for TF-IDF and lifts partly predictable from the query text alone: both query-only routers rank mix_required queries well above the base rate (AUPRC 0.742 and 0.769 against a base rate of 0.254; AUROC 0.892 and 0.910) and reach operating points that neither static policy offers.

Second, the fine-tuned ModernBERT ensemble is a more selective router than the TF-IDF baseline, and this is the statistically established part of the difference between them: its precision on mix_required (0.767 vs. 0.559), its over-routing rate (0.052 vs. 0.155; McNemar on the 144 naive_enough rows, 20 vs. 0 discordant queries, $p < 0.001$), and its routing-label accuracy (0.865 vs. 0.788) are significantly better in paired tests, whereas its higher AUPRC, AUROC, balanced accuracy, and F1 on mix_required from 0.753 to 0.810. The gain on compositional despite the small training set suggests that the encoder may capture signal beyond the lexical patterns available to the TF-IDF baseline. The `bridge_comparison` slice is harder for both routers and is the natural target for follow-up work; on this slice TF-IDF reaches a slightly higher AUPRC (0.418)and its lower recall are differences that a 193-query test set cannot resolve.

Third, this selectivity translates into cost: at its validation-selected threshold the ModernBERT policy reduces mean routed execution time by $42\,\%$ and estimated mean routed model-token usage by $63\,\%$ relative to always-*Mix* (TF-IDF: $30\,\%$ and F1 (0.316) by routing more queries to $52\,\%$; the routed-time difference between the two policies is itself significant, while the token figures are extrapolations from the instrumented subset) while, under the stated assumption about Mix , while ModernBERT keeps higher precision and higher slice accuracy at the cost of missing more *Mix*-beneficial queriescorrectness, retaining about $92\,\%$ (TF-IDF: $94\,\%$) of always-*Mix*'s judged-correct answers.

The type-only diagnostic baseline shows that much Fourth, a large part of the routing signal is already carried by a coarse categorical label: predicting *Mix* for `compositional` questions and *Naive* otherwise already on this benchmark coincides with question-type structure: the non-deployable type-only rule reaches an F1 of 0.667(Table 4.1), because `compositional` questions dominate , ModernBERT exceeds it significantly only on precision (and on routing-label accuracy before, but not after, adjustment for the five heuristic comparisons), TF-IDF does not exceed it significantly on any metric, and the `compositional` slice carries most of the *Mix*-beneficial class on this benchmark. ModernBERT, which sees only the query text and never this benchmark-metadata label, achieves higher point estimates than the categorical rule on F1, precision, and routing-label accuracy. The router should therefore be read as predicting an operational label from phrasing: what it adds over the type-only reference reflects signal the encoder extracts from how questions are worded, and what it does not exceed reflects benchmark structure rather than a deeper property of the queriesqueries.

5.2 Relation to prior work and research contribution

The results sit at the intersection of the three lines of work reviewed in Chapter 2. Adaptive-RAG [4] showed that a small classifier can choose among retrieval-augmented reasoning strategies of increasing cost on the basis of the question alone. The present results show that the same idea carries over to the retrieval layer inside one fixed Hybrid RAG framework, where the choice is between a vector-only and a graph-enhanced retrieval mode, and that it holds under a conservative evaluation in which the labels are judge-derived, the learned router is held against a lexical baseline and a category heuristic, and the comparisons are tested statistically. RouterBench [5] frames routing as a cost–quality trade-off between language models and reports that simple routers recover much of the strongest model's quality at a fraction of its cost; the cost–performance frontier of Section 4.3 reproduces that pattern for retrieval modes, with routed token cost falling by 52–63 % at a loss of roughly six to eight percentage points of retained judged-correct answers.

Stepping back from the routing comparison, the labeling stage is itself informative about the retrieval modesGraphRAG and LightRAG [2, 3] motivate graph-enhanced retrieval by its ability to connect evidence across documents and document its higher cost. The labeling stage of this thesis adds a quantitative view of when that cost pays off on a multi-hop benchmark. Of the 2,000 judged queries, 714 (about 36 %)fell in the none_enough class, where neither *Naive* nor *Mix* was judged beneficial relative to *Naive* on 327 (16 %), *Naive* was already sufficient on 959 (48 %), and neither mode produced a correct answer . These queries are on 714 (36 %). The last group is excluded from the routing dataset because routing between two modes cannot help when both fail, but the size of the class its size is notable in its own right: on a large share of these multi-hop questions the limiting factor is the strength of the retriever, not the choice between its two modes . A stronger retrievalbackbone, lies beyond the choice between the two modes — the failure can sit in retrieval, graph construction, context assembly, generation, or judging — so these queries need improvements to the end-to-end answering pipeline, or an additional fallback strategy, rather than a better router over *Naive* and *Mix*, is what those queries would need .

On the modeling side, the finding that a fine-tuned encoder-only classifier is competitive with, and in precision better than, a lexical baseline while remaining trainable on a single CPU is consistent with the evidence that encoder-only models remain strong classifiers at a fraction of the cost of generative models [15, 16]. The strength of the type-only heuristic qualifies this: part of what the encoder learns on 2WikiMultihopQA is the benchmark's own question-type structure, expressed through phrasing.

The contribution of this thesis is therefore threefold: a focused empirical study of query-only routing between a low-cost vector retrieval mode and a more expensive graph-enhanced mode within a single Hybrid RAG framework, treated as a supervised classification problem with judge-derived labels and evaluated with paired statistical tests; a reproducible pipeline and dataset for that problem; and the specific finding that the advantage of the encoder router over a lexical baseline lies in its precision, and hence in routed cost, rather than in a demonstrably better ranking of which queries need the expensive mode.

5.3 Industrial relevance and societal impact

The benchmark used here is multi-hop by construction, so every query is a comparatively hard case for vector-only retrieval and the `mix_required` rate is correspondingly high. In a deployment such as Ericsson's, where a substantial fraction of queries are single-hop factual look-ups that *Naive* retrieval already handles, more traffic would fall in the `naive_enough` class. An accurate router would then send an even larger share of queries to the low-cost path, so the cost saving reported here is , if anything, a conservative estimate of what query-only routingcould deliver on a less uniformly multi-hop workload.

The per-question-type breakdown and the A workload with more *Naive*-sufficient queries could permit larger savings, although this depends on how well the router transfers and on the class balance and retrieval-cost ratio of the target domain.

In deployment terms, the routing layer studied here is inexpensive to train and to run, although not free to set up. The router itself trains in hours on a CPU and requires no manual annotation, but generating its training labels requires running both retrieval modes and the judge once per training example, and the cost of constructing the graph index is unaffected by routing. At query time it adds no generative-model call; the reported seed-averaged ensemble performs three encoder forward passes per query (a single trained model, or a distilled one, would need one). The router's own inference latency was not included in the routed execution-time measurements and should be measured in a deployed implementation, although it is expected to be small next to retrieval calls taking 11–24 seconds. Its operating point is a threshold that can be moved along the cost–performance frontier of Section 4.3 to match a site's relative cost of an unnecessary *Mix* call against an under-routed query, and the same construction applies to any retrieval stack that exposes a cheap and an expensive mode over the same index. The main practical caveat is the one raised by the type-only diagnostic baseline both rely on the benchmark's per-question category label. In a real deployment that label does not exist; users send free-form queries and the routing layer sees only the query text. The category-based slices are useful as a diagnostic device because they show which subpopulations the router handles well and which it does not, and they help separate signal that comes from coarse structural cues (such as whether the question is a compositional multi-hop lookup) from signal that comes from finer phrasing. They are not a deployment recipe; the learned routers approximate the same structure from the query text alone, which is the only input available outside the benchmarkheuristic: the router should be validated on the target workload, since part of the signal learned on a benchmark may be benchmark structure.

The societal relevance lies in making grounded question answering over large document collections cheaper to operate. Lower compute per query lowers the operating cost of such systems and may reduce their energy footprint (Section 5.5), and it makes richer retrieval affordable for the queries that need it. The same mechanism carries a risk that is discussed next: a system that decides per query how much effort to spend can serve some kinds of question systematically worse than others.

5.4 Ethical considerations

The data are public benchmark questions constructed from Wikipedia; no personal or user data were processed, and no Ericsson-internal documents were used in the experiments. The routing labels are produced by an LLM-as-a-Judge that has access to the gold answer, a methodologically reasonable but imperfect protocol whose limitations are discussed in Section 5.6.2; the thesis is explicit throughout about not treating these labels as ground truth, and validating a sample of judge decisions against human judgments is left to future work.

The same framing helps interpret the contrast between the `compositional` and `bridge_comparison` slices. The fact that ModernBERT is strong on `compositional` and weaker on `bridge_comparison` indicates which phrasings the encoder has learned to recognize on this benchmark, and does not extend to main ethical consideration concerns the routing mechanism itself. Building a system that decides on a claim that `compositional` queries are easier to route in production. Within this benchmark, per-query basis which retrieval path to use could in principle interact with downstream notions of answer quality in ways that are hard to anticipate. If the router systematically routes some kind of question to the `compositional` slice contains more rows and the rows share a more consistent surface form. lower-cost path, that kind of question could end up consistently worse-served, even when the average behavior of the system improves. The per-question-type breakdown in Section 4.4 is one way to surface this kind of differential behavior, and any future work that scales this approach to user-facing settings should keep similar per-segment analyses in view.

Always-

5.5 Sustainability

This thesis is centered on making a Hybrid RAG system more resource-efficient. The most direct connection is to the United Nations (UN) Sustainable Development Goal (SDG) number 12 (responsible consumption and production). Graph-enhanced retrieval is substantially more compute-intensive than vector retrieval [3, 12], and a router that avoids applying it to queries that do not need it reduces measured compute-related cost proxies such as routed token usage and execution time. This may reduce associated energy use in deployment, although energy consumption was not measured directly. There is also a softer connection to SDG number 9 (industry, innovation, and infrastructure), in that adaptive routing is a building block for industrial RAG deployments such as the one motivating this work at Ericsson, where serving company-internal documentation at scale is a real operational concern.

5.6 Limitations

The conclusions of this thesis hold within the scope described in Section 1.5, and the following limitations qualify them. The corresponding avenues for future work are collected in Section 6.2.

5.6.1 Token-cost measurement scope

Model-token cost in this thesis is estimated using per-mode mean token usage from an instrumented subset of $N = 15$ queries spanning all four 2WikiMultihopQA question types. This subset provides a preliminary estimate of per-mode mean token usage, and full per-query logging across the $1{,}286$-row routing dataset would be needed for exact routed token costs and uncertainty estimates. The reported token costs should therefore be read as estimated routed token usage from per-mode means, and not as exact measured token usage for every held-out query. Two further caveats apply. Applying one global mean per mode assumes that token usage within a mode is unrelated to which queries a router selects; if harder questions produce systematically longer contexts and outputs, the *Mix* illustrates why the two result tables must be read together. It is label-incorrect on *Naive*-sufficient rows because it uses more retrieval than necessary, although the underlying *Mix* answer on those rows can still be correct. The absence of a direct final-answer-accuracy comparison for each routed policy is treated as a limitation queries a selective router actually routes need not match a generic *Mix* sample's mean. And the instrumented subset consists of the first 15 queries of the processing stream — it spans all four question types but was not sampled at random (the identities are published with the code). The reported token savings are accordingly preliminary estimates.

5.6.2 Judge labels

Routing labels come from an LLM-as-a-Judge and are used as judged supervision, not as absolute ground truth. The judge sees the gold answer and the same prompt is used consistently across the dataset [18], but judge errors may still propagate into the labels. The risk is largest for borderline answers, partially correct answers, and answers that use different surface forms from the gold answer. In addition, because the same model checkpoint produces the candidate answers and judges them, the judge may share biases with the generated answers; comparing its labels against a different judge model or human annotations, as outlined in Section 5.6.36.2, would quantify this risk.

The reported metrics are still informative for adaptive routing. Routing-label F1 measures whether the router selects the minimum judged sufficient mode. Mean routed time and estimated token usage measure the cost of doing so. The under-routing and over-routing rates separate answer-risk errorsfrom cost-waste errors. Together they describe how often the router agrees with the judge and how expensive that agreement islabel `mix_required` should be read operationally: it captures cases in which, under the chosen LightRAG configuration and judge prompt, *Mix* was judged correct when *Naive* was not, and it does not isolate which component of *Mix* caused the improvement. *Mix* differs from *Naive* in graph traversal, context assembly, and retrieval breadth, and the label aggregates over all of these.

5.6.3 Final answer accuracy of routed policies

The offline policy evaluation reports routing-label performance and routed cost. It does not independently re-judge the final answer selected by each routed policy. This matters because false-positive *Mix* decisions are counted as routing-label errors, although the *Mix* answer for the same query may still be correct. False-negative *Mix* decisions are the errors most directly associated with answer-quality risk, because the router selects *Naive* on a query where *Mix* was judged necessary.

The

5.6.4 Generalizability

The experiments use one Hybrid RAG framework (LightRAG), two of its retrieval modes (*Naive* and *Mix*), and one public benchmark (2WikiMultihopQA). The held-out test set has 193 queries. This is large rows, which is enough to compute meaningful bootstrap intervals and too small not enough to resolve small differences between the routerswith confidence. When the ModernBERT ensemble is bootstrapped on the same test set as TF-IDF, the two routers' $95\,\%$ intervals overlap on every threshold-independent metric: the AUPRC intervals ($[0.610, 0.853]$ for TF-IDF and $[0.645, 0.889]$ for ModernBERT) overlap heavily, and the same holds for AUROCand balanced routing-label accuracy. The routing-label-accuracy intervals ($[0.731, 0.845]$ for TF-IDF and $[0.819, 0.912]$ for ModernBERT) come closest to separation but still overlap. The point estimates therefore favour ModernBERT consistently, yet none of the individual gaps is statistically significant at this sample size. A larger benchmark would be needed before stronger claims can be maderouters. The conclusions therefore apply most directly to the studied setting. Extending the routing labels to industrial corpora (for example Ericsson internal documentation) was considered but is blocked by the absence of per-query gold answers, which is what made the judge-labeling protocol possible on 2WikiMultihopQA in the first place.

Finally, the cost picture is sharper in model tokens than in execution time. The two static baselines bracket a mean routed range of 10.95 to 24.42 seconds per query in execution time, and 3,458 to 17,857 tokens per query in model-token usage. The learned routers select operating points around 14 to 17 seconds per query (a 30–42 % reduction relative to always-*Mix*) and 6,700 to 8,500 tokens per query (a 52–63 % reduction relative to always-*Mix*), at non-trivial The strong type-only diagnostic baseline also indicates that part of the routing signal may be benchmark-specific. The results should not be read as evidence that the same router would transfer unchanged to arbitrary industrial queries without additional validation. Two further selection effects qualify the results. Because the benchmark was chosen partly for producing a comparatively balanced routing-label distribution in preliminary runs (Section 3.1.1), the reported classifier performance may overstate what would be obtained on workloads with more extreme class imbalance. And because the train/validation/test split is row-random rather than grouped by relation template or source entities, near-duplicate phrasings of the same question template can occur on both sides of the split, so part of the apparent query-only predictability may come from recognizing shared surface templates rather than from generalizing to unseen phrasings.

5.6.5 Statistical power

The test split has 193 queries, of which 49 are `mix_required`F1. The token reduction is larger than the time reduction because *Mix* adds one extra LLM call per query and . As Table 4.2

shows, this is enough to establish differences of the size observed in precision and routing-label accuracy, but the paired-bootstrap intervals for the AUPRC, AUROC, and balanced-accuracy differences are about ± 0.03 to ± 0.08 wide, so differences of a few hundredths on these metrics cannot be resolved, and the per-question-type comparisons are underpowered (the `bridge_comparison` slice has eight positives). Read as minimum detectable differences at $80\,\%$ power and $\alpha = 0.05$, the paired standard errors implied by Table 4.2 correspond to roughly 0.04 for AUROC, 0.08 for balanced accuracy, 0.11 for AUPRC, and 0.13 for F1, well above the observed gaps of 0.018, 0.018, 0.027, and 0.068. Non-significant differences should therefore be read as undecided rather than as evidence of equality, and the statements about ranking quality in this thesis are deliberately phrased as point-estimate observations.

5.6.6 Routing beyond the query

The router used in this thesis is query-only by construction, so it cannot react to what retrieval actually surfaces. A router that also has access to some lightweight signal from the corpus or from a low-cost first retrieval pass could in principle make a better decision, at the cost of a slightly more expensive routing layer; Section 6.2 outlines such a design, which would be the natural way to attack the harder `bridge_comparison` slice.

5.6.7 Calibration and abstention

The router currently outputs a single thresholded decision, and the calibration of the ModernBERT router's softmax outputs was not investigated. Without calibrated probabilities the router cannot abstain on low-confidence queries or trade cost against confidence in a substantially larger assembled context [3, 12], while execution time is partly absorbed by network latency that does not scale with token volume. The routing decision itself contributes zero LLM tokens because neither router calls a generative model, and its inference cost is expected to be negligible relative to retrieval cost on either dimension. principled way, and the threshold has to be chosen from a validation sweep as was done here.

5.7 Use of generative AI tools

Two generative AI coding assistants, OpenAI Codex and Anthropic's Claude Code, were used during this project in an advisory role. They served as discussion partners rather than as an author: the author used them to talk through how to articulate an argument or phrase a passage of the report, to discuss alternative ways of implementing parts of the pipeline and the router code, and to locate errors during debugging. They were used in the same way when preparing the revisions in response to supervisor and examiner feedback. The research question, the experimental design, the experiments, the analyses, and the conclusions are the author's own. The tools did not run experiments or produce results on their own; analysis code drafted with their help was reviewed and executed by the author, every suggestion concerning wording or code was reviewed, edited, and accepted or rejected by the author, and all reported numbers were checked against the underlying logs and output files. The author takes full responsibility for the content of this thesis. The generative language model that is part of the method (the Gemini model used to generate answers and to judge them) is a component of the experimental pipeline described in Chapter 3 and is not the subject of this section.

In the author's experience the main benefit of these tools was speed of iteration: having a competent interlocutor available at any time shortened the loop between an idea, a draft, and a critique of that draft, and made it easier to consider several formulations or implementations before committing to one. The main risk is over-reliance: an assistant can produce a fluent explanation or a plausible piece of code that is wrong in detail, so claims about the data, the code, or the literature have to be checked against the actual logs, sources, and outputs, which was done throughout this work. Used in this way, as professional discussion partners whose output is always verified, the tools supported the work without replacing the author's judgment.

Chapter 6

Conclusions and future work

This chapter summarizes the conclusions from the experiments answers the research question on the basis of the results of Chapter 4 and their discussion in Chapter 5 (Section 6.1), discusses the main limitations and natural avenues and collects the directions for future work (Section 5.6), and reflects on the broader context of the project that follow from the study's limitations (Section 5.46.2).

6.1 Conclusions

This thesis studied whether the choice between LightRAG's low-cost *Naive* retrieval mode and its more expensive *Mix* retrieval mode can be predicted from the query text alone on the 2WikiMultihopQA benchmark using a lightweight encoder-only classifier instead of a large language-model controller. Routing labels were derived by running both retrieval modes on each benchmark question of the 2,000 selected benchmark questions and asking an LLM-as-a-Judge [18] to compare each mode's answer against the gold answer. The label `mix_required` is operational: it captures whether, under the chosen LightRAG configuration and judge prompt, *Mix* All classification and answer-retention results are therefore conditional on the binary routing dataset, i.e. on the 64 % of judged queries for which at least one of the two modes was judged correctwhen *Naive* was not, and it does not isolate which component of *Mix* drove the improvement; a deployed router also receives queries on which both paths fail, for which routing determines cost but cannot improve answer quality.

The research question asked, within a fixed Hybrid RAG setting, how well a lightweight query-only classifier predicts a judge-derived routing label for the minimum sufficient retrieval mode, and how using that classifier as an offline routing policy affects routed cost relative to static *Naive* and *Mix* baselines. The remainder of this section answers the two sub-questions in turn.

The first sub-question concerns classifier accuracy on the routing label. On the held-out test split of 193 queries, the ModernBERT ensemble achieves higher point estimates than the TF-IDF baseline on every aggregate routing-label metric: AUPRC (0.769 vs. 0.742), balanced routing-label accuracy (0.802 vs. 0.784), F1 on the minority `mix_required` class (0.717 vs. 0.650), and routing-label accuracy (0.865 vs. 0.788). When both routers are bootstrapped In paired statistical tests on the same test set, however, their 95 % intervals overlap on every threshold-independent metric, so none of these gaps is statistically significant at 193 test queries; the routing-label-accuracy gap is the widest and comes closest to separation. The practical queries, the gains in precision (0.767 vs. 0.559, $p < 0.001$) and routing-label accuracy ($p = 0.006$; Holm-adjusted $p = 0.035$) are significant, whereas the gains in AUPRC, AUROC, balanced accuracy, and F1 are not. The answer to the first sub-question is therefore that the fine-tuned encoder predicts the routing label at least as well as is the more selective router: it is significantly more precise and more accurate on the routing label than the TF-IDF baseline, and consistently a little better in point estimate, but has higher but not significantly higher point estimates on AUPRC, AUROC, balanced accuracy, and F1, and has a lower but not significantly lower recall on `mix_required`; the present test set is too small to certify a strict improvementestablish that it also ranks the queries better. The two routers place themselves at opposite ends of the precision-recall precision–recall trade-off on the `mix_required` class: TF-IDF has lower under-routing (0.057 vs. 0.083; 6 vs. 1 discordant queries, not significant), and ModernBERT has significantly lower over-routing (0.052 vs. 0.155; McNemar $p < 0.001$). ModernBERT also beats exceeds the non-deployable type-only heuristic — which predicts *Mix* for `compositional` questions and *Naive* otherwise, reaching F1 0.667 and routing-label accuracy 0.803 — significantly on precision, on F1, precision, and routing-label accuracy before but not after adjustment for the five heuristic comparisons, and non-significantly on F1, using only the query text rather than benchmark metadata; TF-IDF does not differ significantly from the heuristic on any metric.

The second sub-question concerns routed cost relative to static baselines. Both learned routers reach test-set operating points around 14 to 17 seconds of mean routed execution time, compared with 10.95 seconds for always-*Naive* and 24.42 seconds for always-*Mix*, and achieve higher routing-label F1 than either static policy at substantially lower mean routed time than always-*Mix*. Measured in model tokens, they route at roughly $6{,}700$ to $8{,}500$ tokens per query, compared with $3{,}458$ for always-*Naive* and $17{,}857$ for always-*Mix*. The selected ModernBERT operating point reduces estimated routed token cost by 63 % relative to always-*Mix*, and the selected TF-IDF operating point by 52 %. The answer to the second sub-question is therefore that query-only routing recovers most removes a large part of the cost of always-*Mix* — up to a 42 % reduction in mean time and a 63 % reduction in estimated tokens for the ModernBERT ensemble — while, under the stated assumption that *Mix* remains correct on queries where *Naive* was judged sufficient, retaining about 92 % of always-*Mix*'s judged-correct answers. The routing decision itself adds negligible cost on either dimension because neither router calls a generative model; ModernBERT's single encoder forward pass no generative-model call; the reported ModernBERT ensemble performs three encoder forward passes per query, which costs more than computing the TF-IDF features and logistic-regression score, but both are negligible and neither router's inference latency was included in the routed-time measurements — an expected, but not separately measured, small overhead next to the retrieval and generation they gate.

Taken together, the two answers indicate that the *Naive*-sufficient versus *Mix*-beneficial distinction is partly predictable from query text alone in this setting, and that a lightweight encoder classifier extracts that predictability at least as much of that predictability well as a TF-IDF logistic-regression baseline — a little more in point estimate, though within overlapping confidence intervals significantly more on the cost-relevant precision side, with higher but unresolved point estimates on ranking quality, balanced accuracy, and F1, and a lower recall that was likewise not statistically resolved — and more than exceeds a coarse question-category heuristic significantly on precision. The reported accuracy values are routing-label accuracies, and they are distinct from independently measured final answer accuracies. Within that scope, the result supports learned query routing as a promising cost-control layer for Hybrid RAG: the router can often identify when the expensive path is unnecessary, and it therefore reduces routed cost without always defaulting to the most expensive retrieval mode.

6.2 Future work

Several directions follow directly from the limitations discussed in Section 5.6.

Model-token cost in this thesis is estimated using per-mode mean token usage from an instrumented subset of $N = 150$ queries spanning all four 2WikiMultihopQA question types. This subset provides a preliminary estimate of per-mode mean token usage , and full per-query logging across

6.2.1 Per-query token logging

Logging model-token usage for every query in the 1,286-row routing datasetwould be needed for exact routed token costs and uncertainty estimates. The reported token costs should therefore be read as , rather than for the $N = 15$ instrumented subset, would turn the estimated routed token usage from per-mode means, and not as exact measured token usage for every held-out query.

Per-query token logging would also costs into measured ones, make it possible to report bootstrap intervals for token cost on the same footing as for execution time, and to study allow studying whether token usage varies systematically across question types or routing labels.

Routing labels come from an LLM-as-a-Judge and are used as judged supervision, not as absolute ground truth. The judge sees the gold answer and the same prompt is used consistently across the dataset [18], but judge errors may still propagate into the labels. The risk is largest for borderline answers, partially correct answers, and answers that use different surface forms from the gold answer. In addition, because the same model checkpoint produces the candidate answers and judges them, the judge may share biases with the generated answers; comparing its labels against a different judge model or human annotations, as outlined above, would quantify this risk.

The label `mix_required` should be read operationally: it captures cases in which, under the chosen LightRAG configuration and judge prompt, *Mix* was judged correct when *Naive* was not, and it does not isolate which component of *Mix* caused the improvement. *Mix* differs from *Naive* in graph traversal, context assembly, and retrieval breadth, and the label aggregates over all of these.

Follow-up work would include validating a sample of judge decisions manually, comparing multiple judge models, and tightening the

6.2.2 Judge validation

A sample of judge decisions should be validated manually, and the labels compared against those of at least one judge model from a different model family than the generator, so that the risk of shared biases between generator and judge (Section 5.6.2) can be quantified. The answer-normalization rules where possible. could be tightened where the comparison reveals systematic disagreements.

The offline policy evaluation reports routing-label performance and routed cost. It does not independently re-judge the final answer selected by each routed policy. This matters because false-positive *Mix* decisions are counted as routing-label errors, although the *Mix* answer for the same query may still be correct. False-negative *Mix* decisions are the errors most directly associated with answer-quality risk, because the router selects *Naive* on a query where *Mix* was judged necessary.

6.2.3 Re-judging the answers of routed policies

A future evaluation should retain per-mode correctness indicators for every query, or re-run the judge on the answer selected by each policy, so that judged final answer accuracy can be reported alongside routed cost without the assumption about *Mix* correctness used in Section 4.5. With per-policy answer correctness available, the main trade-off would become answer correctness against token cost.

The experiments use one Hybrid RAG framework (LightRAG), two of its retrieval modes (*Naive* and *Mix*), and one public benchmark (2WikiMultihopQA). The held-out test set has 193 rows, which is enough to compute bootstrap intervals and not enough to resolve small differences between routers. The conclusions therefore apply most directly to the studied setting.

6.2.4 Other frameworks, benchmarks, and a larger test set

Applying the same methodology to other Hybrid RAG frameworks (for example GraphRAG [2] or hybrid retrieval modes in other systems) and to other multi-hop benchmarks (HotpotQA, MuSiQue [9, 10]) is a the natural way to test how robust the routing signal is . Extending the routing labels and how much of it is benchmark structure. A test set several times larger than the present one would be needed to resolve the ranking-quality differences left open in Section 4.1. Extending the approach to industrial corpora (for example Ericssoninternal documentation) was considered but is blocked by the absence of per-query gold answers , which is what made the judge-labeling protocol possible on 2WikiMultihopQA in the first place.

The strong type-only diagnostic baseline also indicates that part of the routing signal may be benchmark-specific. The results should not be read as evidence that the same router would transfer unchanged to arbitrary industrial queries without additional validationsuch as Ericsson's internal documentation would require either a small human-labeled set of questions with reference answers or a judge protocol that does not depend on gold answers. The large `none_enough` class also motivates improving the end-to-end answering pipeline itself, or adding a fallback strategy, so that more of the queries on which both modes currently fail become routable.

6.2.5 Routing beyond the query

The router used in this thesis is query-only by construction. A router that also has access to some lightweight signal from the corpus or from a low-cost first retrieval pass could in principle make a better decision, at the cost of a slightly more expensive routing layer. A A natural follow-up is to compare the query-only router against a two-stage scheme that first runs the *Naive* mode, inspects whether the retrieved chunks contain the entities referenced in the query, and escalates to *Mix* only when this signal looks weak. This is similar to how Adaptive-RAG conditions on inexpensive intermediate signals at the language-model level [4]. The routing layer would remain inexpensive (one round of vector retrieval instead of an expensive graph traversal) and could materially improve the harder slices, in particular `bridge_comparison`.

6.2.6 Calibration and abstention

The router currently outputs a single thresholded decision. Routing under cost constraints is a natural setting for selective prediction [28] and calibrated probabilities [29], where the router could abstain on low-confidence queries and either escalate to a more expensive but more reliable controller or run both retrieval modes and combine the answers. The Investigating the calibration of the ModernBERT router's softmax outputs was not investigated in this work and is a natural follow-upis the first step in that direction.

This thesis is centered on making a Hybrid RAG system more resource-efficient. The most direct connection is to the UN SDG number 12 (responsible consumption and production). Graph-enhanced retrieval is significantly more compute-intensive than vector retrieval [3, 12], and a router that avoids applying it to queries that do not need it reduces measured compute-related cost proxies such as routed token usage and execution time. This may reduce associated energy use in deployment, although energy consumption was not measured directly. There is also a softer connection to SDG number 9 (industry, innovation, and infrastructure), in that adaptive routing is a building block for industrial RAG deployments such as the one motivating this work at Ericsson, where serving company-internal documentation at scale is a real operational concern.

The work has some ethical aspects worth flagging. The routing labels are produced by an LLM-as-a-Judge that has access to the gold answer. This is a methodologically reasonable but imperfect protocol. It is acceptable for the present study for two reasons: the judge decides a constrained, reference-grounded comparison against the gold answer rather than an open-ended quality score, which limits how far it can drift from a correct reading; and the prediction target is deliberately the judge's operational routing decision, not a human's notion of answer quality. Validating a sample of judge decisions against human judgments, and measuring how often the two disagree, is left to future work, and the thesis is explicit throughout about not treating these labels as ground truth. Building a system that decides on a per-query basis which retrieval path to use could in principle interact with downstream notions of answer quality in ways that are hard to anticipate. If the router systematically routes some kind of question to the lower-cost path, that kind of question could end up consistently worse-served, even when the average behavior of the system improves. The per-question-type breakdown in Section 4.1 is one way to surface this kind of differential behavior, and any future work that scales this approach to user-facing settings should keep similar per-segment analyses in view.

References

[1] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2312.10997

[2] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A Graph RAG approach to query-focused summarization," 2025. [Online]. Available: https://arxiv.org/abs/2404.16130

[3] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," 2025. [Online]. Available: https://arxiv.org/abs/2410.05779

[4] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024. doi: 10.18653/v1/2024.naacl-long.389 pp. 7036–7050. [Online]. Available: https://aclanthology.org/2024.naacl-long.389/

[5] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, "RouterBench: A benchmark for Multi-LLM routing system," 2024. [Online]. Available: https://arxiv.org/abs/2403.12031

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," 2023. [Online]. Available: https://arxiv.org/abs/2310.11511

[7] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proceedings of the 28th International Conference on Computational Linguistics*, D. Scott, N. Bel, and C. Zong, Eds. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020. doi: 10.18653/v1/2020.coling-main.580 pp. 6609–6625. [Online]. Available: https://aclanthology.org/2020.coling-main.580/

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[9] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018. doi: 10.18653/v1/D18-1259 pp. 2369–2380. [Online]. Available: https://aclanthology.org/D18-1259/

[10] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022. doi: 10.1162/tacl_a_00475. [Online]. Available: https://aclanthology.org/2022.tacl-1.31/

[11] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, B. Webber, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020. doi: 10.18653/v1/2020.emnlp-main.550 pp. 6769–6781. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550/

[12] L. Stamatescu, "GraphRAG costs explained: What you need to know," Microsoft Foundry Blog, Aug. 2024, accessed 2026-05-04. [Online]. Available: https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/graphrag-costs-explained-what-you-need-to-know/4207978

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008. [Online]. Available: https://papers.nips.cc/paper/7181-attention-is-all-you-need

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019. doi: 10.18653/v1/N19-1423 pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423/

[15] A. Benayas, M. A. Sicilia, and M. Mora-Cantallops, "A comparative analysis of encoder only and decoder only models in intent classification and sentiment analysis: navigating the trade-offs in model size and performance," *Language Resources and Evaluation*, vol. 59, no. 3, pp. 2007–2030, 2024. doi: 10.1007/s10579-024-09796-y. [Online]. Available: https://doi.org/10.1007/s10579-024-09796-y

[16] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," 2024. [Online]. Available: https://arxiv.org/abs/2412.13663

[17] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988. doi: 10.1016/0306-4573(88)90021-0

[18] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[19] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Mathematical Programming*, vol. 45, no. 1, pp. 503–528, 1989. doi: 10.1007/BF01589116

[21] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[22] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015. doi: 10.1371/journal.pone.0118432. [Online]. Available: https://doi.org/10.1371/journal.pone.0118432

[23] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998. doi: 10.1162/089976698300017197

[24] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. doi: 10.1214/aos/1176344552

[25] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach," *Biometrics*, vol. 44, no. 3, pp. 837–845, 1988. doi: 10.2307/2531595

[26] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. doi: 10.1007/BF02295996

[27] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[28] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[29] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

Appendix A

Source code

The implementation code, the data-preparation pipeline, the training scripts, and the evaluation scripts used in this thesis are publicly available at `https://github.com/Dantecool02/Thesis_ericsson`. ; the version corresponding to this thesis is tagged `examiner-revision`. The repository's `results/` directory additionally publishes the routing labels of all $2{,}000$ judged queries, the exact train/validation/test membership of the binary routing dataset, the identities of the instrumented token subset, the summary outputs behind every reported number, and checksums of the underlying run artifacts.

Appendix B

Experimental configuration

This appendix lists the configuration used to produce the routing labels, train the routers, and compute the reported routed-cost estimates. Table B.1 summarizes the LightRAG, data, and judge settings. Table B.2 summarizes the router training settings. Section B.1 maps each pipeline stage onto a script in the public code repository (Appendix A), and Section B.2 reproduces the judge prompt verbatim.

Table B.1: LightRAG, data, and judge configuration used in this thesis. Each row names one configurable component of the pipeline and gives the exact value used in all reported experiments. *Item* is the name of the component (model checkpoint, framework version, retrieval parameter, or instrumentation choice); *Value* is the literal value passed to the corresponding API or library. The same generator model and embedding model are used during indexing and at query time. All Gemini calls use a temperature of 0.0.

Item	**Value**
Generator model	`gemini-3-flash-preview` (Gemini API)
Judge model	`gemini-3-flash-preview` (Gemini API)
Embedding model	`models/gemini-embedding-001` (768-dim)
LightRAG version	`1.4.10` (vendored fork in `external/LightRAG`)
LightRAG chunk token size	1 200 tokens
LightRAG chunk overlap	100 tokens
Naive retrieval parameters	`top_k=35`, `chunk_top_k=35`
Mix retrieval parameters	`mode="mix"`, `chunk_top_k=35`; entity-relation graph built during indexing
LightRAG tokenizer	`gpt-4o-mini` via `tiktoken`
Generator temperature	0.0
Judge temperature	0.0
Token logging method	Gemini API `usage_metadata` total per generation call (prompt, output, and reasoning tokens); embedding-call inputs estimated with tiktoken and excluded from reported totals
Token measurement subset	$N = 150$ queriesspanning all four 2WikiMultihopQA question types $N = 15$ queries: the first 15 of the query stream (all four question types represented; same `top_k=5` as the main runs; identities in `results/token_subset.jsonl`)
Query selection	first 2,000 of the 2,279 queryable train-split samples, in benchmark order
Judge prompt location	Section B.2

Table B.2: Router training configuration used in this thesis. Each row names one router hyperparameter or training-pipeline setting and gives the exact value used in all reported experiments. *Item* is the name of the hyperparameter or setting; *Value* is the value passed to scikit-learn (for the TF-IDF baseline) or Hugging Face Transformers (for the ModernBERT router). All three ModernBERT seeds use the values listed here; only the random seed differs across the three runs.

Item	**Value**
TF-IDF features	Unigrams and bigrams, lowercase, English stop-words removed, min document frequency 2, vocabulary cap 20 000
TF-IDF classifier	Logistic regression, L_2 regularization with $C = 1.0$, balanced class weights, scikit-learn L-BFGS solver, max 1 000 iterations
ModernBERT checkpoint	`answerdotai/ModernBERT-base`
ModernBERT max sequence length	128 tokens
ModernBERT learning rate	2×10^{-5}
ModernBERT weight decay	0.01
ModernBERT batch size	Per-device 8, gradient accumulation 2, effective batch 16
ModernBERT optimizer	AdamW with linear warm-up over $10\,\%$ of steps and linear decay; gradient clip 1.0
ModernBERT max epochs	8, early stopping on validation $\mathrm{F1}_{\mathrm{mix}}$ with patience 3, minimum 3 epochs
ModernBERT seeds	7, 13, 42
Split sizes	900 train / 193 validation / 193 test
Split strategy	Joint stratification by question type and routing label
Hardware	Single CPU; no GPU used for ModernBERT training or inference

B.1 Pipeline scripts

The eight pipeline stages used in this thesis map one-to-one onto scripts in the public code repository (Appendix A). Table B.3 lists the script for each stage in the order it is executed. Running the scripts in this orderreproduces the , with the exact commands documented in the repository's README, regenerates the data, the routers, and the figures used in this thesis from the raw 2WikiMultihopQA download; the caption of Table B.3 states the limits of this claim, and the labels and splits actually used are published in the repository's `results/` directory.

B.2 Judge prompt

Each judged query is sent to the judge model as two messages: a fixed system prompt, and a user prompt that defines the three class labels, states the decision rules, and carries the question, the gold answer, and the two candidate answers in a JSON JavaScript Object Notation (JSON) payload. Both prompts are reproduced verbatim below, so the label definitions and decision rules shown here are part of the prompt the judge actually sees. The judge returns exactly one of the three class labels.

The system prompt:

```
You are a strict evaluator for a retrieval-routing
   dataset.

You must output exactly one class label and nothing else
   .

Allowed labels:
- naive_enough
- mix_required
- none_enough
```

The user prompt, where `{payload}` is the per-query JSON JSON object with the fields `question`, `ground_truth_answer`, `naive_answer`, and `mix_answer`:

```
Classify this example into exactly one of these labels:

- naive_enough: the Naive answer was already enough to
   answer the question correctly.
```

```
- mix_required: the Naive answer was not enough, but the
    Mix answer provided more context that led to a
   correct answer.
- none_enough: neither answer was enough.

Rules:
1. Output exactly one label and nothing else.
2. Do not output JSON.
3. Do not explain your choice.
4. Use the ground-truth answer to judge correctness when
    it is provided.
5. A longer answer is not better just because it is
   longer.
6. If both Naive and Mix are correct, choose
   naive_enough.
7. If Naive is wrong or insufficient and Mix is correct,
    choose mix_required.
8. If both are wrong, insufficient, or clearly fail to
   answer, choose none_enough.
9. Treat explicit admissions of missing information and
   obvious error messages as not enough.

Example:
{payload}
```

Table B.3: Pipeline stages and the script that implements each stage. *Stage* names a logical step of the data and modelling modeling pipeline (sample preparation, indexing, dual-mode execution, token instrumentation, judging, dataset construction, router training, evaluation, or figure generation). *Script* is the path of the Python file in the public code repository (Appendix A) that implements the stage; all paths are relative to the repository root. Running the scripts in the order shownreproduces , with the dataexact commands documented in the repository's README, routersregenerates the pipeline from the raw 2WikiMultihopQA download; the API-dependent stages (indexing, querying, judging) cannot be guaranteed to reproduce bit-identical model outputs, which is why the labels, splits, and figures instrumented-subset identities actually used are published in this thesis the repository's `results/` directory, from which the raw 2WikiMultihopQA downloaddeterministic training and analysis stages (fixed seeds) can be re-run exactly.

Stage	**Script**
Prepare 2WikiMultihopQA samples	`src/dataset_tools/prepare_2wiki_samples.py` `src/dataset_tools/prepare_2wiki_samples.py`
Index the corpus in LightRAG	`src/data_synthesis/index_2wiki_lightrag.py` `src/data_synthesis/index_2wiki_lightrag.py`
Run both retrieval modes on every queryable sample the first 2,000 queryable samples	`src/data_synthesis/query_lightrag.py` `src/data_synthesis/query_lightrag.py`
Instrument token usage on the $N = 150$ $N = 15$ subset	`src/data_synthesis/query_lightrag_token_usage.py` `src/data_synthesis/query_lightrag_token_usage.py`
Generate routing labels with LLM-as-a-Judge	`src/data_synthesis/judge_query_results.py` `src/data_synthesis/judge_query_results.py`
Build the binary routing dataset and joint-stratified splits	`src/router/prepare_router_dataset.py` `src/router/prepare_router_dataset.py`
Train the TF-IDF baseline router	`src/router/train_baseline_router.py` `src/router/train_baseline_router.py`
Train the ModernBERT routers (seeds 7, 13, 42)	`src/router/train_modernbert_router.py` `src/router/train_modernbert_router.py`
Run held-out classification, routed-cost simulation, per-question-type breakdown, and bootstrap CIs for the seed-averaged ensemble Final ensemble evaluation: classification, routed cost,	`src/router/thesis_analysis_ensemble.py` `src/router/thesis_analysis_ensemble.py`

Stockholm, Sweden 2026

www.kth.se